面向最优风险决策的 NIPT 时点选择模型构建与胎儿健康判定

摘　要

本文围绕无创产前检测 NIPT，基于检测数据，探究不同孕妇指标对 NIPT 的影响，结合临床实际提出科学合理的判定方法，为尽早判定胎儿健康状况，降低孕妇潜在风险提供解决方法，并分析相关检验误差对结果的影响。

问题一中，先对原始数据预处理，对检验孕周进行格式转换，统一数值型数据为浮点数，过滤异常 GC 含量等指标，最后筛选出有效数据 936 条。为了筛选出与 Y 染色体浓度显著相关的指标，利用**皮尔逊相关系数**与热力图，得到 7 个显著相关的指标。通过中心化处理共线性，构建**多元线性回归模型**，通过**岭回归**求解系数。模型结果证实 BMI 对 Y 染色体浓度存在**负向非线性关联**、孕周对 Y 染色体浓度存在**正向相关**。参考基因组比对比例和唯一比对读段数对 Y 染色体浓度存在负向关联、被过滤掉读段数的比例和重复读段比例对 Y 染色体浓度存在正向关联，但这四个指标相关强度较弱。再通过构建**混合效应模型**和 Gamma 回归模型解决残差问题，从整体显著性来看，混合效应模型似然比为 2205.76，优于 Gamma 回归模型，其残差检验的 Durbin-Watson 统计量接近 2，说明残差**无自相关**；受高 BMI 群体影响，残差存在非正态。虽然受多因素影响岭回归模型 R^2=5.30%，但核心变量**无多重共线性**，系数估计可靠。

对于问题二，减少 BMI 差异造成的 NIPT 检测误差，划分区间寻找该组区间最优时点。对 BMI 数据分布统计，发现不同区间样本数差异极大。采用**动态相邻互补算法**对 BMI 单区间进行均衡分组，最终形成 **5 个**组内同质性高的组别。通过 **DBSCAN 聚类**去噪，提高数据质量，将数据集利用分组算法划分组别。建立**风险损失函数**量化检测失败风险与时间风险，结合网格搜索求解最佳时点。结果显示 BMI 越高，最佳时点越晚且风险越高，BMI 最低组时点为 **10.9** 周，最高 BMI 组时点为 **13.4** 周。通过**蒙特卡洛模拟**检测误差对结果的影响，高 BMI 组受误差影响更显著，风险增加率达 7.71%，模拟达标率均值下降至 72.4%。

对于问题三，先是构造了与达标时间对比的相关性矩阵，搭建变量方差膨胀因子 VIF 评估变量间的多重共线性程度，识别出变量间极高的多重共线性。通过**主成分分析 PCA** 对高共线性变量做降维处理，保留 9 个解释了 94.6%方差的主成分。使用 **K-means 聚类**划分 BMI 为**低中高三个组别**，计算达标时间统计量。发现高 BMI 组的达标时间显著延长，且风险显著高于低 BMI 组。加入误差评分，对数据二次分组进行计算。误差亚组结果显示高 BMI 孕妇达标时间变化更加明显，在高误差的条件下，达标率对比原始结果下降 17.6%，得出误差对高 BMI 孕妇的检测影响较大。

对于问题四，综合 X 染色体上特征因素构建能判定女胎异常的模型。对原始数据预处理与特征工程，处理怀孕次数表达，转换部分特征为二值，并构建新特征。针对样本数差异大，采用允许样本不均衡基于树的**集成学习模型**。通过交叉验证的方式保证结果公正，并利用网格搜索最优参数，在高召回率阈值和 F2 最优阈值下训练模型令漏诊风险最小，贴合临床错判不漏判的原则。特征重要性结果一致，X 染色体浓度标准化和 CG 偏离度时最关键影响因素。全面评估三个模型指标对比 Z 值传统规则，其结果表明三个模型均优于 Z 值传统规则。其中 **GradientBoosting** 模型最优，准确率达 **86.1%**，召回率到 **76.5%**，F2 分数有 0.631，对比 Z 值传统异常检出能力提高 550%。将异常概率值的界限设为 **0.122**，若待测数据在模型中超过此值，则判定为异常。

关键词：NIPT　混合效应模型　DBSCAN　主成分分析 PCA　K-means 聚类　集成学习

一、 问题重述

1.1 问题的背景

NIPT 即为无创产前检测，通过采集母体血液，检测胎儿的游离 DNA 片段、分析胎儿染色体是否存在异常，从而评估胎儿患某些疾病的风险。在临床中，畸形胎儿主要有唐氏综合征、爱德华氏综合征和帕陶氏综合征，三种体征分别由 21 号、18 号和 13 号“染色体游离 DNA 片段的比例”即染色体浓度是否异常决定。NIPT 的准确性主要由胎儿性染色体浓度判断，男胎为 XY，女胎为 XX。孕期在 10 周~25 周之间可以检测胎儿性染色体浓度，如果男胎的 Y 染色体浓度达到或高于 4%、女胎的 X 染色体浓度没有异常，则可认为 NIPT 的结果是基本准确的，否则难以保证结果准确性要求。随着孕周的增大，孕妇需要承担的风险就越大。因此，NIPT 检测能降低许多不必要的风险。实际中应尽早发现不健康的胎儿，否则会带来治疗窗口期缩短的风险，早期 12 周以内风险较低；中期 13－27 周风险高；晚期 28 周以后风险极高。

1.2 问题的提出

实践表明，男胎 Y 染色体浓度与孕妇周数及其身体质量指数 BMI 紧密相关。孕妇之间存在个体差异，简单的经验分组和统一的检测时间点，会对 NIPT 的准确性产生较大的影响。因此，对不同的 BMI 值的孕妇进行分组，确定每组最佳的 NIPT 的时间点，能够减少因胎儿不健康而缩短治疗窗口期带来的潜在风险。为了研究各类孕妇群体合适的 NIPT 时点，并对检测的准确性进行分析，根据附件中孕妇的 NIPT 数据建立数学模型研究下列问题：

问题一：分析胎儿 Y 染色体浓度与孕妇的孕周数和 BMI 等指标，给出相应的关系模型，同时检验其显著性。

问题二：已知男胎孕妇 BMI 是影响胎儿 Y 染色体浓度的最早达标时间的主要因素。对男胎孕妇的 BMI 分组，给出每组的区间以及最佳的 NIPT 时间点，令孕妇可能的潜在风险最小，并分析检测误差对结果的影响。

问题三：男胎 Y 染色体浓度达标时间受身高、体重、年龄等多种因素影响，综合考虑这些因素、检测误差和胎儿的 Y 染色体浓度达标比例，根据男胎孕妇的 BMI，进行分组并确定最佳的 NIPT 时点，令孕妇可能的潜在风险最小，并分析检测误差对结果的影响。

问题四：女胎及孕妇不携带 Y 染色体，尝试利用女胎孕妇的 21 号、18 号和 13 号染色体非整倍体（AB 列）作判定结果，综合考虑 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素，给出女胎异常的判定方法。

二、 问题分析

2.1 问题一的分析

对于问题一，目标为分析 Y 染色体浓度与孕妇相关指标的相关性，给出关系模型并检验显著性。按模型所需对原始数据进行预处理，包括数值转换，异常或缺失值处理，构建特征工程等。计算相关性系数，能够帮助筛选出显著相关指标。因为要求分析关系以及检验显著性，初步判定这是一个回归分析问题，构建回归模型。再对模型进行统计检验，确保其结果有一定的可靠性。后续延伸分析可补充指标影响力排序，同时对结果可视化直观展示问题结果。

2.2 问题二的分析

对于问题二，目标寻找最优 BMI 分组以及每组最佳 NIPT 时间点，令综合风险最小化，同时评估检测误差对结果的影响。为科学合理对 BMI 分组，要先了解真实的分布情况，确保分组时样本量不能过少且尽量均衡同时组内特征相近，组间有差异。建立模型求解最佳时点，构建综合风险模型，能够描述时间风险与失败风险的目标函数，对目标函数寻求最优解得到每个分组最佳时点，验证模型确保结果需要能够抵抗检测误差的影响。

2.3 问题三的分析

对于问题三，在问题二的基础上考虑身高、年龄、检测误差等多种因素对 Y 染色体浓度达标时间的影响，给出 BMI 分组以及最佳时点。利用相关性分析找出关键因素，再通过线性回归量化各因素的独立影响。解答问题二了解，因素可能共同影响，需要处理多重共线问题。通过对检测数据评估，误差评分函数量化检测误差的程度，构造新特。需令分组结果有同质性，使用聚类算法对其分组。定义风险函数，在分组间搜索最优解得最佳时点。分析误差对结果得影响，在不同检测误差程度下对达标时间和最佳时点造成影响。

2.4 问题四的分析

对于问题四，核心在于建立模型利用女胎孕妇的 21 号、18 号和 13 号染色体非整倍体作判定结果，能够判定女胎是否异常，可归属于二分类问题。对数据做特征工程用于训练模型。由于样本量差异较大，选用模型应当能够有针对性，在样本量少下也能保证一定精度。孕妇可能在不同的时间多次检测，按孕妇分组保证一孕妇所有数据仅在测试集或训练集。同时模型应有可解释性，能够提供实际的决策依据及建议。

三、 模型假设

1. 假设不同孕妇的检测数据相互独立
2. Y 染色体浓度为 4%作为准确临界值
3. 假设临床特征与染色体异常风险存在统计相关性
4. 假设染色体异常与特征之间存在非线性关系[9]

四、 符号说明

符号	说明
NIPT	无创产前检测
BMI	身体质量指数
β_j	回归系数
X_j	变量
Y	Y 染色体浓度
β_0	常数项
VIF	方差膨胀因子
R^2	模型决定系数
$AdjR^2$	调整决定系数
P_{pass}	达标率
FF	游离 DNA 浓度

SS_{res}	残差平方和
SS_{tot}	总平方和
SS_{reg}	回归平方和
t	统计量
std	标准误差
err	与标准误差相关的统计量
p	概率值

五、 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 数据预处理

根据题目的要求及模型所需，对原始数据进行预处理，令数据能用于后续模型计算。先定义孕周转换函数，识别检测孕周列中的字符串将其转换为以数值型描述的孕周，孕周字符串常见于三种格式‘w+天数’、‘w’、‘周+天数’，将周和天转换按周为单位的十进制数值。读取数据后定义核心指标列用于模型构建。应让数据统一且符合临床标准，清洗数据并过滤异常值，确保数据的完整性和质量。先用孕周转换函数令字符串数据转为数值型数据，筛选符合临床标准的数据，将 Y 染色体浓度范围限定在 0-0.2；孕妇 BMI 范围限定在 18-50；孕周范围限定在 10-25 周；GC 含量限定在 30%-70%；比对比例以及读数段设置合理的上下限。再将核心指标列的数值类型统一为浮点型。胎儿健康状态分为是与否，有且仅有这两种可能情况下，将其映射为 1 和 0。该过程对 1082 条原始数据预处理后，得到有效数据 936 条，过滤比例为 13.49%，绝大多数的数据正确可靠。[13]

5.1.2 模型的建立

问题一，要求分析 Y 染色体浓度与孕妇相关指标的相关性，给出关系模型并检验显著性。FF 作为 NIPT 技术有效检测胎儿染色体数目异常的重要参数，可能会受到母体因素的影响，如 BMI、年龄及妊娠方式等。[1]先定义所有的分析变量，包括孕周，孕妇 BMI 等指标。[14]对所有变量计算与 Y 染色体浓度的皮尔逊相关系数，以及 p 值验证其显著性。计算结果如下表：

表 1 相关性分析

指标	相关系数	p 值	显著
孕周	0.080884	0.0133	是
BMI	-0.159460	0.0000	是
GC 含量	-0.007342	0.8225	否
参考基因组比对比例	-0.111605	0.0006	是
过滤读段比例	0.084217	0.0099	是
log（原始读段数）	-0.090861	0.0054	是
重复读段比例	0.078069	0.0169	是
唯一对比读段数	-0.091700	0.0050	是

依据公认的 $p<0.05$ 则可认定其显著性，从表中能够得到 7 个显著相关的指标。BMI 显示出最强的负相关性，这表明体重指数高的孕妇其 Y 染色体浓度倾向于较低。[11]除了计算皮尔逊相关系数外，绘制相关性热力图分析相关性。

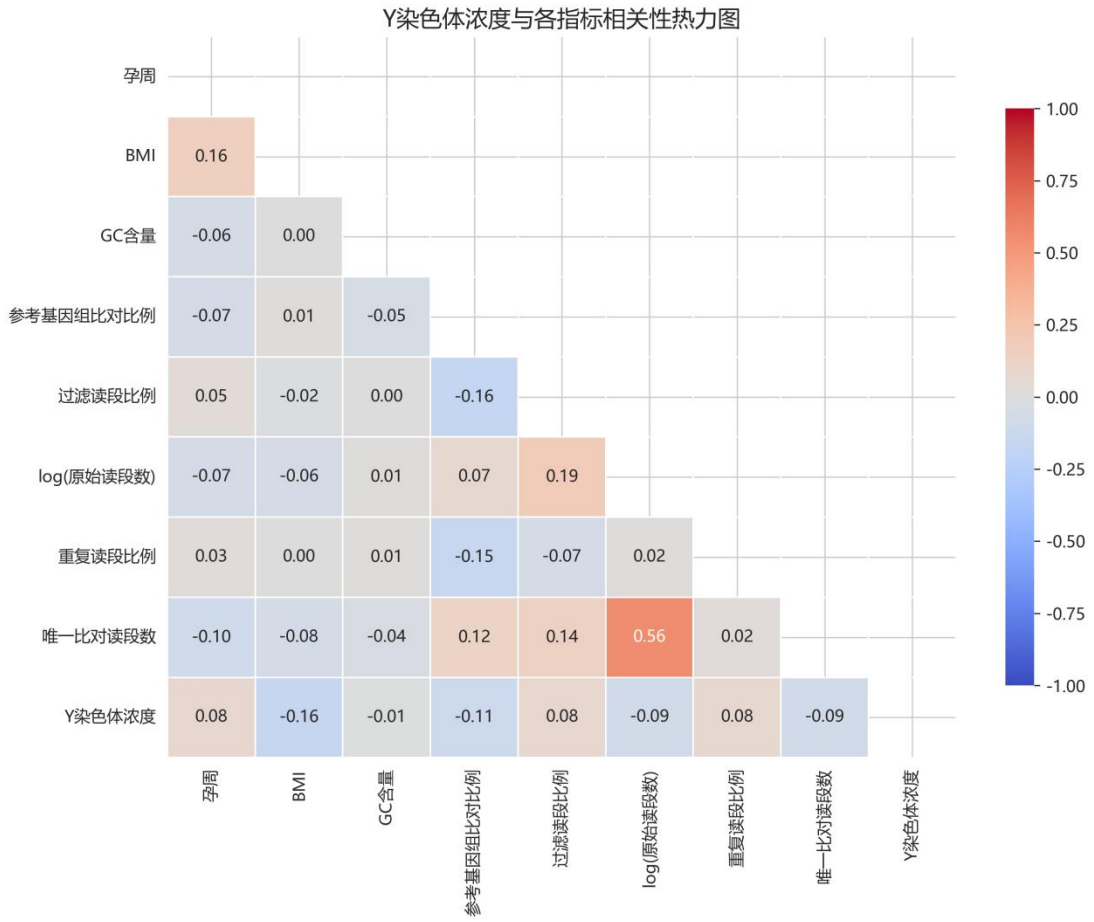


图 1 Y 染色体浓度与各指标热力图[12]

按照问题分析，判定为回归问题，建立回归模型。变量之间可能存在多重共线性问题，对连续变量 BMI 和孕周进行中心化处理减少共线性的影响。可能存在两个变量共同作用影响，变量之间不一定全是线性，使用中心化后的变量构建交互项、二次项到模型中，能够提升模型的严谨性以及预测精度。交互项捕捉变量之间的协同效应，考虑到受到两个及以上的变量共同影响；变量之间不全是存在线性关系的，二次项能够处理非线性关系的变量；同时，某些数据尺度的差异很大，极端值对模型贡献弊大于利，取对数的处理能令其相对稳定。这里选取孕周与 BMI 作为交互项，BMI 设二次项。构建模型时仅保留显著相关且低共线性的变量，使用岭回归估计系数。选取中心化后的孕周、BMI，重复读段比例，过滤读段比例做变量建立多元线性回归模型。多元线性模型的公式如下[2]：

$$Y = \beta_0 + \beta_1 \widetilde{X_1} + \beta_2 \widetilde{X_2} + \beta_3 \widetilde{X_2^2} + \beta_4 X_4 + \beta_5 X_5 + \varepsilon \quad (1)$$

注：β_0 为常数项，代表所有变量为 0 时，Y 染色体浓度的基准值，因为自变量不会为 0 所以实际上无直接生物学意义，仅用于统计拟合。β_1~β_5 是回归系数，X_j 是自变量，ε为误差项。

5.1.3 模型的求解

求解模型的核心是回归系数的估计，这里采用岭回归估计回归系数，以解决多重共线性问题。计算输出模型统计量包括模型决定系数，调整决定系数以及回归系数表。统计计量两者的计算公式如下：

$$R^2=1-\frac{SS_{\text{res}}}{SS_{\text{tot}}}=\frac{SS_{\text{reg}}}{SS_{\text{tot}}} \quad (2)$$

R^2 表示模型解释变异数据的程度，越接近 1 模型拟合度越好；SS_{res} 是残差平方和，SS_{tot} 是总平方和，SS_{reg} 是回归平方和。

$$A\text{djusted}R^2=1-\frac{(1-R^2)(n-1)}{n-p-1} \quad (3)$$

调整 R^2 是考虑自变量数目对 R^2 的影响，n 是样本量，p 是自变量的个数。
结果为下列表：

表 2 模型统计计量

指标	值
模型决定系数	0.0530
调整决定系数	0.0479

上述表中的计算结果 R^2 的值为 0.0530，调整 R^2 的值为 0.0479。模型能够解释 5% 的变异具有一定的意义，表示所选的变量对 Y 染色体浓度有一定的预测能力。

依据题目核心要求量化 Y 染色体浓度与指标间的关系，结果输出为回归系数表，包含系数值用于量化关系，当自变量增加一个单位，因变量平均变化；标准误衡量系数估计值的不确定性和波动性，数值越小说明估值越准确、可靠；t 值检验该自变量的系数是否显著不等于 0 的证据的统计量，简单理解为绝对值越大时自变量越重要；设计问题核心的 p 值，用于判断关键指标是否具有统计显著性。输出结果如下表：

表 3 回归系数表

自变量	系数值	标准化系数
孕周（中心化）	0.000941	0.115256
BMI（中心化）	-0.001437	-0.131629
BMI 二次项	-0.000229	-0.133354
重复读段比例	0.005892	0.000513
过滤读段比例	0.007619	0.000827

根据岭回归模型结果，BMI 及其二次项是影响 Y 染色体浓度的最重要因素，显示 BMI 与 Y 浓度呈负相关且存在非线性递减效应，孕周影响强度低于前两者并呈稳定正相关；而两个测序质量指标重复读段比例和过滤读段比例可能因为存在严重多重共线性问题，独立影响强度被稀释，导致标准化系数均接近零，表明实际影响微弱。该结果提示在临床实践中应重点考虑孕妇 BMI 和孕周对 Y 染色体浓度的联合影响。

题目提到过为了检测的准确性，部分孕妇存在多次检测的行为，该行为可能导致残差聚集性，同时普通多元回归无法处理个体差异，因此构建混合效应模型[3]。将孕妇代码作为分组变量，同时捕捉固定效应以及随机效应，前者为孕周、BMI 的影响，后者为孕妇个体差异。输出模型结果，如表：

表 4 混合效应模型结果

coef	std	err	t	P>|t|	[0.025	0.0975]
const	0.070	0.008	8.590	0.000	0.054	0.086
孕周（中心化）	0.003	0.000	16.584	0.000	0.003	0.003
BMI（中心化）	-0.001	0.001	-1.507	0.132	-0.002	0.000
BMI 二次项	-0.000	0.000	-1.772	0.076	-0.000	0.000
重复读段比例	0.069	0.217	0.318	0.750	-0.356	0.494

过滤读段比例	0.259	0.173	1.459	0.135	-0.081	0.599
Group Var	0.001	0.006	/	/	/	/

中心化变量孕周 p<0.001，表示极显著，可理解为孕周越大，Y 染色体浓度越高；其余变量均呈现不存在统计显著性或边缘显著，说明这些变量对 Y 染色体浓度的影响缺乏足够的证据支持。组间方差值为 0.001，标准误为 0.006，这说明了不用孕妇之间，Y 染色体浓度存在固有变异，同时验证使用混合效应模型的合理性。

Y 染色体浓度属于正偏态数据，能够依靠 Gamma 回归解决残差非正态，打破残差正态的限制。输出 Gamma 回归的结果：

表 5 Gamma 回归结果

coef	std	err	t	P>|t|	[0.025	0.0975]
const	19.3845	2.149	9.018	0.000	15.172	23.597
孕周（中心化）	-0.1467	0.043	-3.450	0.001	-0.230	-0.063
BMI（中心化）	0.2776	0.069	4.051	0.000	0.143	0.412
BMI 二次项	0.0668	0.014	4.631	0.000	0.039	0.095
重复读段比例	-139.2235	57.869	-2.406	0.016	-252.645	-25.802
过滤读段比例	-114.7850	47.532	-2.415	0.016	-207.946	-21.624

Gamma 回归的系数需通过链接函数转化理解，系数的符号直接反映对 Y 染色体浓度倒数的影响。中心化变量 BMI 和 BMI 二次项对 Y 染色体浓度有显著正向影响，明确 BMI 越高，Y 染色体浓度越低，且说明 BMI 对 Y 染色体浓度的负向影响存在非线性强化效应，随着 BMI 升高，其导致 Y 染色体浓度降低的幅度会进一步加大；重复读段比例和过滤读段比例对 Y 染色体浓度有负向影响，推测被过滤的读段中包含更多干扰信号，过滤后浓度估计更准确。

为确定模型的合理可靠性，对模型从拟合优度、整体显著性、共线性风险、残差特性四个维度检验。结果均以图表展示。

表 6 拟合优度检验

岭回归模型解释 Y 染色体浓度变异比例	5.30%
混合效应模型对数似然值	2205.7626
Gamma 回归模型伪 R^2	-0.0199
Gamma 回归模型似然比	1941.13

岭回归模型仅能解释 5.30% 的 Y 染色体浓度变异，其对因变量的解释力较弱；Gamma 回归的伪 R^2，提示该模型不适合刻画当前数据规律。

表 7 整体显著性检验

混合效应模型对数似然比	2205.76
Gamma 回归模型似然比	1941.13

对数似然值反映模型对因变量变异的整体解释能力混合效应模型的对数似然值显著高于 Gamma 回归，对数似然值越大表明模型拟合效果越好，因此混合效应模型在三类模型中对数据的适配性更优。

计算方差膨胀因子 VIF，经过中心化后孕周和 BMI 的 VIF 值降至 1.03 和 1.16，这表明不存在多重共线性，模型系数估计稳定，不会出现系数偏大或符号异常的情况。

绘制模型残差诊断图分析。

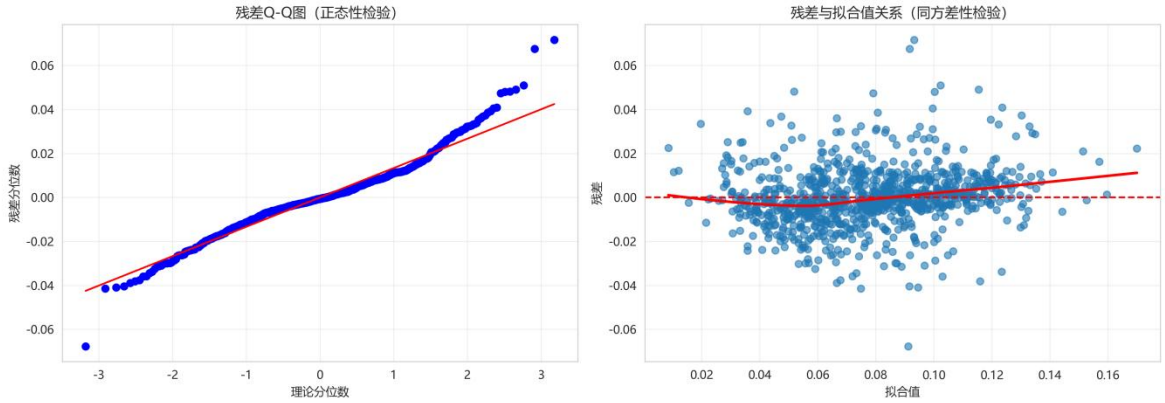


图 2 残差诊断图

从上图可得知残差不满足正态分布假设，存在尾部偏离正态的情况，可能存在异方差性，但偏离程度相对温和。对混合效应模型的残差正态性检验，其 p 值为 0.0000，远小于 0.5 的阈值，说明残差显著偏离正态分布，同时解释为何 Gamma 回归有针对性却效果不佳。残差独立性检验的统计量在 2.1084 处于 1.5-2.5 的阈值区间内，证明残差间无自相关，保证模型系数估计的可靠性。

5.2 问题二模型的建立与求解

对于问题二，要求对 BMI 进行合理分组，给出区间以及最佳时点且令潜在风险最小。先通过下列流程图展示问题二模型建立求解的全过程。

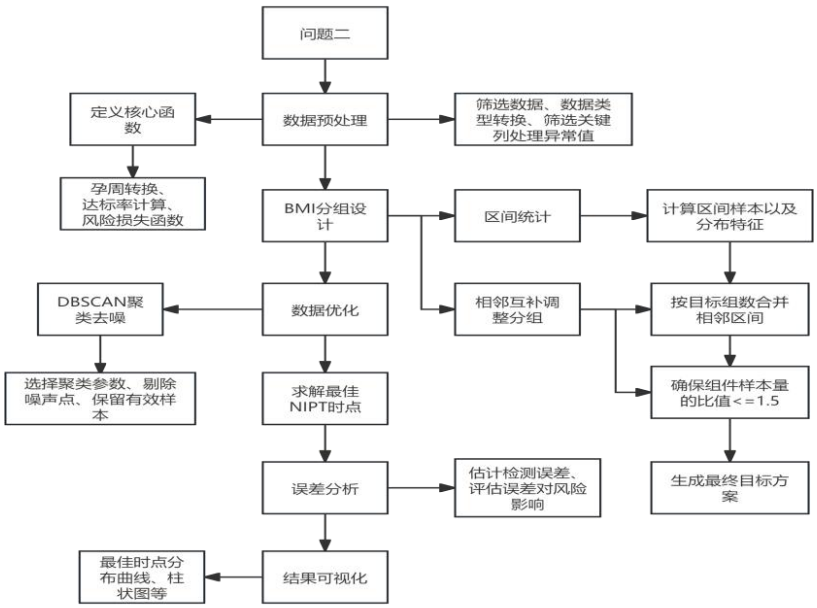


图 3 问题二整体流程图

首先设定所需的函数，包括孕周转换函数、达标率计算函数、风险损失函数。孕周转换函数延续问题一里的方法。达标率计算函数核心任务时计算目标孕周附近的一个时间窗口，Y 染色体的浓度达标的比例，计算时先确定时间窗口，检查窗口内是否有足够的样本，筛选样本然后计算达标样本得出达标率，公式如下：

$$P_{pass}=\frac{\text{时间窗口内}Y\text{染色体浓度}\geq 0.4\text{的样本数}}{\text{时间窗口内总样本数}} \tag{4}$$

风险损失函数可视为融合了临床情况的决策模型，其核心在于平衡检测过早导致的失败风险与过晚导致的时间风险，时间风险在 12 周前处于基础风险值 1，12~25 周风险每周增加 0.2，25 周过后惩罚性高风险值 10；失败风险计算考虑达标率同时设置惩罚机制。权重系数分别为 0.6、0.4，说明实际中及时检测的重要性。风险损失函数表达式为：

$$\text{风险损失} = 0.6\times\text{时间风险} + 0.4\times\text{失败风险} \tag{5}$$

数据预处理大部分参考问题一一样处理，不同点在于异常值过滤部分，这里考虑 BMI 范围在 20~50 间，Y 染色体浓度在 0.01~0.2 间，以及在参考基因组上比对的比例大于 0.7。为了能够科学合理分组，对 BMI 单单位区间统计，探索其分布情况，下表展示其统计特征：

表 8 BMI 分组特征

有效区间	22
最小样本数	1
最大样本数	185
平均	48.32
样本量标准差	59.4

呈现出不同 BMI 区间的样本量差异极大，分布极度不均。进一步呈现 BMI 区间具体的分布情况，绘制柱状图。

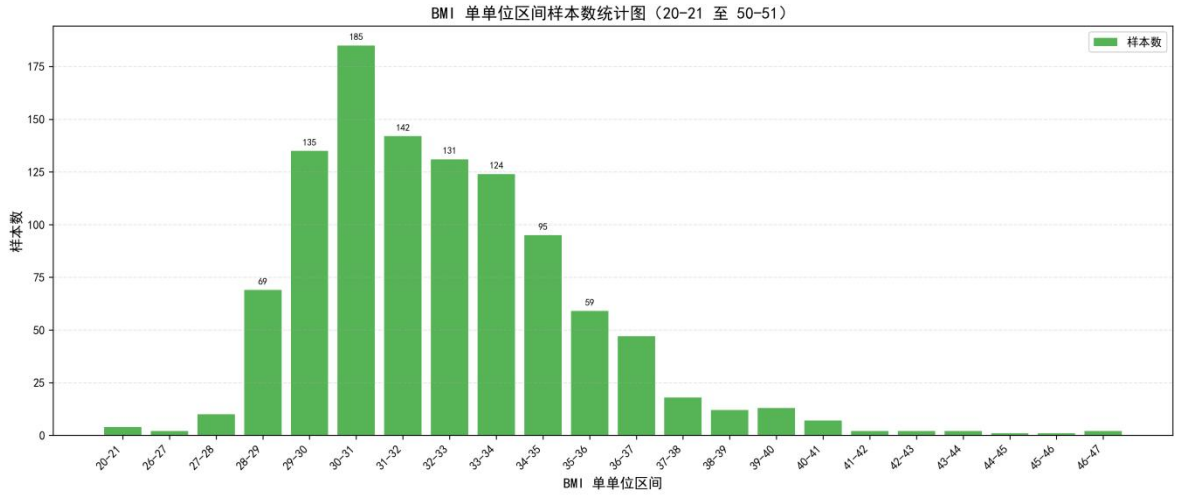


图 4 BMI 单区间样本统计图

可以观察到样本集中在 30-35 的区间，40 区间以上样本量几乎为 0，差异最大的两个区间样本相差 184 个样本，某些单 BMI 区间的样本量无法独立支撑独立统计分析，因此通过相邻互补调整合并区间，实现组间样本量均衡。

设计 BMI 分组算法，以平衡样本量，组内同质性高，组外有所差异作为目标。输入单单位区间样本数序列，从低 BMI 到高 BMI 遍历区间，动态合并相邻区间，直到样本量接近目标平均值。同时设定约束条件来确保均衡性，组间最大样本量比值小于等于 1.5，避免任何一组独大或过小。算法应具有实用性，在生成分组边界值后，参考临床关键阈值调整。验证分组是否符合同质性，计算组内 BMI 变异系数，均达到要求小于 0.1，证明组内同质性极高。分组算法输出五个分组[20,30)，[30,31)，[31,34)，[34,36)，36+，。

通过 DBSCAN 聚类去噪[4]，选择聚类特征并进行标准化处理，明确识别并剔除异常点，最后移除异常样本，提高数据质量。其功能工作流程如图：

DBSCAN 使用欧氏距离来衡量样本点之间的相似性。对于样本点 x 与 y 计算欧氏距离为：

$$d(x,y)=\sqrt{\sum_{i=1}^{n}(x_i-y_i)^2} \tag{6}$$

根据分组算法对数据集分组，创造出新列并计算这些分组的稳定性指标如表：

表 9 分组情况

BMI 分组	样本量	均值	变异系数	12 周达标率	Y 浓度均值
[20,30)	220	28.90	0.045	0.769	0.081
[30,31)	185	30.48	0.009	0.875	0.080
[31,34)	397	32.42	0.026	0.842	0.078
[34,36)	154	34.83	0.017	0.714	0.074
36+	107	38.22	0.061	0.000	0.056

变异系数均小于 0.1，组内同质性高，样本量相对均衡，这样的结果表示分组很成功。

分组后就要求解每组最佳时点，即构建优化风险决策模型。对每个分组在孕周 10~25 区间内，以 0.1 周为步长进行网格搜索，对每个候选点计算该点下的达标率，根据约束条件，只考虑达标率大于 0.7 的点，如果满足约束时点，计算综合风险，选取综合风险最小的点作为此组最佳的 NIPT 时点。

表 10 求解结果

BMI 分组	最佳时点(周)	风险损失	达标率	是否满足约束
[20,30)	10.9	0.600	100.0%	TRUE
[30,31)	12.1	0.645	91.7%	TRUE
[31,34)	11.8	0.600	100.0%	TRUE
[34,36)	12.4	0.648	100.0%	TRUE
36+	13.4	0.848	80.0%	TRUE

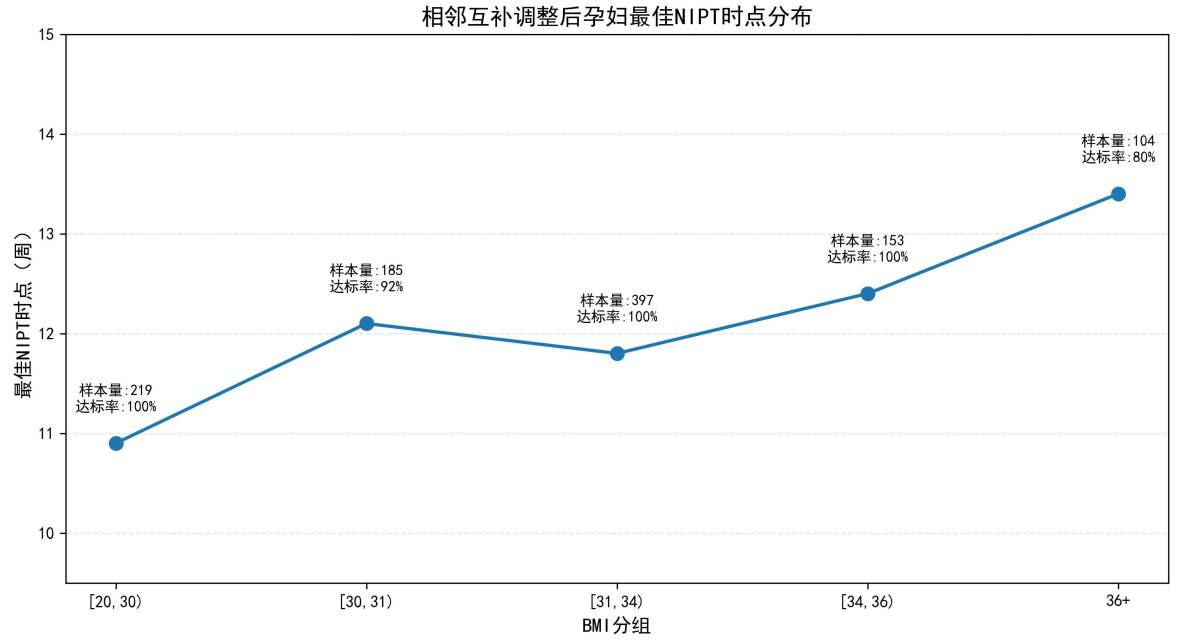


图 5 相邻互补调整后孕妇最佳 NIPT 时点分布

成功为每个分组找到最优解。BMI 越高，最佳检测时点越晚，且风险随之升高，完全符合临床认知。可视化呈现不同 BMI 分组的 Y 染色体浓度分布规律，如图 6：

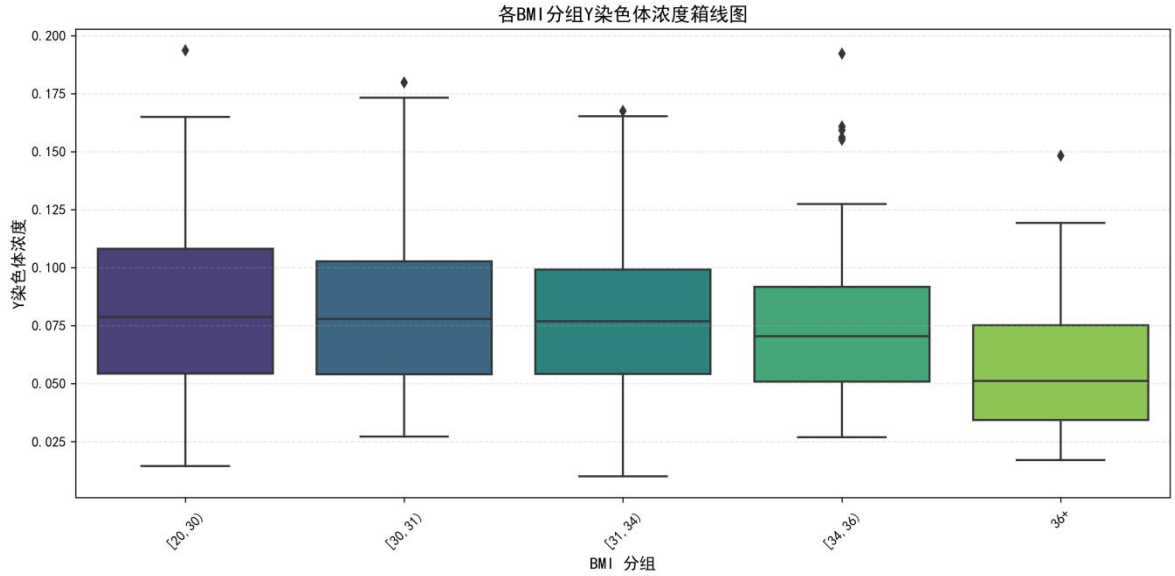


图 6 BMI 染色体浓度箱线图

随之孕妇 BMI 的升高，Y 染色体的浓度有所下降，侧面验证 BMI 高的 NIPT 最佳时点晚于 BMI 低的情况，体现模型结果的合理性。另外，HUDEVOVA 等曾在 2014 年报道，游离染色体浓度越低，异常染色体越不容易检出，因游离染色体浓度低造成 NIPT 失败的人群胎儿非整倍体的风险更高。[5]

最后分析检测误差对结果的影响。通过重复测量的孕妇数据估计检测误差，量化 Y 染色体浓度检测的误差大小，得到检测误差标准差为 0.0051。检测误差估计公式如下：

$$\text{误差} = \sqrt{\frac{1}{N}\sum_{i=1}^{n}(X_i - \bar{X})^2} \tag{7}$$

通过蒙特卡洛 1000 次模拟进行带有随机误差的检测[6]，评估误差对结果的影响。随机误差为真实数值与均值为 0，检测误差作标准差的正态分布函数。其结果如下：

表 11 误差组与原结果对比

BMI 分组	风险损失	模拟风险均值	风险增加率	模拟达标率均值
[20,30)	0.600	0.612	1.95	0.971
[30,31)	0.645	0.639	-0.97	0.933
[31,34)	0.600	0.636	6.03	0.910
[34,36)	0.648	0.657	1.40	0.977
36+	0.848	0.913	7.71	0.724

高 BMI 组受误差影响更显著，风险损失和模拟风险均值为五个组中最高，风险增加率达 7.71%，模拟达标率均值下降至 72.4%。通过柱状图更直观的展示误差的影响的程度。

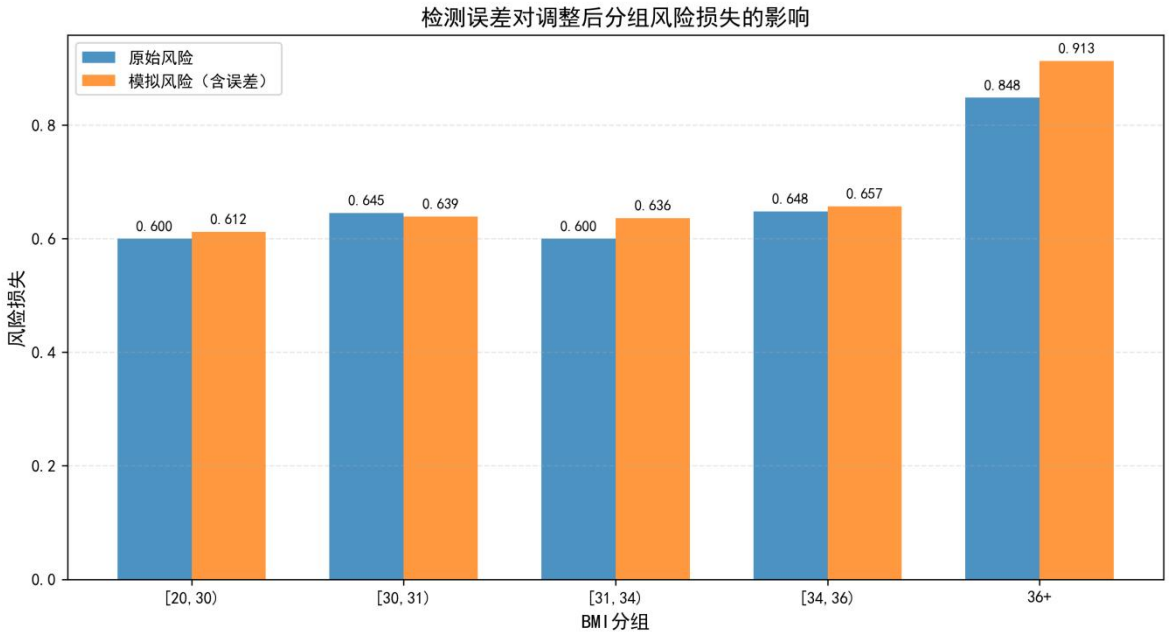


图 7 误差影响差异

5.3 问题三模型的建立与求解

对于问题三，明确不同个体间 FF 水平存在差异，任何潜在干扰母体或胎盘中游离 DNA 浓度的因素，都可能进一步影响 NIPT 结果的准确性。[1]综合考虑身高、年龄、检测误差等因素对 Y 染色体浓度影响下，划分 BMI 组，求解最佳时点，分析误差影响。数据处理延用前面得转换方法，新增 Y 染色体浓度达标时间计算与误差评分体系。获取每个孕妇检测 Y 染色体浓度达标的最早周数，筛选孕妇数据，对数据清洗，得到有达标时间的数据后构造误差评分体系。该体系评分越高代表本次检测的误差越大，数据质量越不可靠。误差评分由四个部分相加得到，计算形式如下：

$$\text{误差评分} = A + B + (1 - C) + D \times 0.1 \tag{8}$$

注：A 为重复读段的比例，B 为被过滤掉读段数的比例，C 为在参考基因上比对的比列，D 为 GC 异常。

评分没有依赖单一指标，从数据冗余度、数据质量、数据特异性和数据分布四个为维度综合评估数据质量，一定程度上覆盖 NIPT 检测当中常见的误差来源，但仍有局限性。因为误差的来源不仅只有母体，亚健康的胎儿也会影响胎儿游离 DNA 的浓度，干扰 NIPT 的检测。[1]后续用作影响因素放入模型以及作亚组变量进行亚组分析。最后对数据去重处理，每个孕妇仅保留一条记录，数据处理部分结束。

为了在众多影响因素当中识别关键因素，对数据做统计分析。计算影响因素与 Y 染色体浓度达标时间的相关系数，初步筛选潜在影响因素，影响因素输出按相关性排序。结果如下：

表 12 各因素与 Y 染色体浓度达标时间相关系数表

达标时间	1.000000
体重	0.247039
孕妇 BMI	0.200002
身高	0.140509
重复读段数比列	0.100291
GC 含量	0.059640

年龄	0.057206
被过滤读段数比列	0.045083
在参考基因组上比对的比列	0.022474
误差评分	0.007140
唯一比对的读段数	-0.125276
原始读段数	-0.127922
Y 染色体浓度	-0.233837

从中得到孕妇 BMI 显示出较强的正相关性，表明 BMI 高的孕妇需更长的孕周才能使得 Y 染色体浓度达标；原始读段数和唯一比对读段数与达标时间的负相关性证实了数据量多且有效对早期检测的重要性。

通过方差膨胀因子 VIF 检测变量之间是否存在多重共线性，评估回归模型中自变量的多重共线程度，避免因严重的多重共线问题导致模型不稳定。计算每个影响因素的 VIF，并识别到了变量之间极高的多重共线性，VIF 值高达数万，一般高于 5 时就可视作存在严重的多重共线问题。通过 PCA 降维处理高共线特征[7]，绘制 PCA 方差解释率图，显示各主成分的贡献和累积方差。

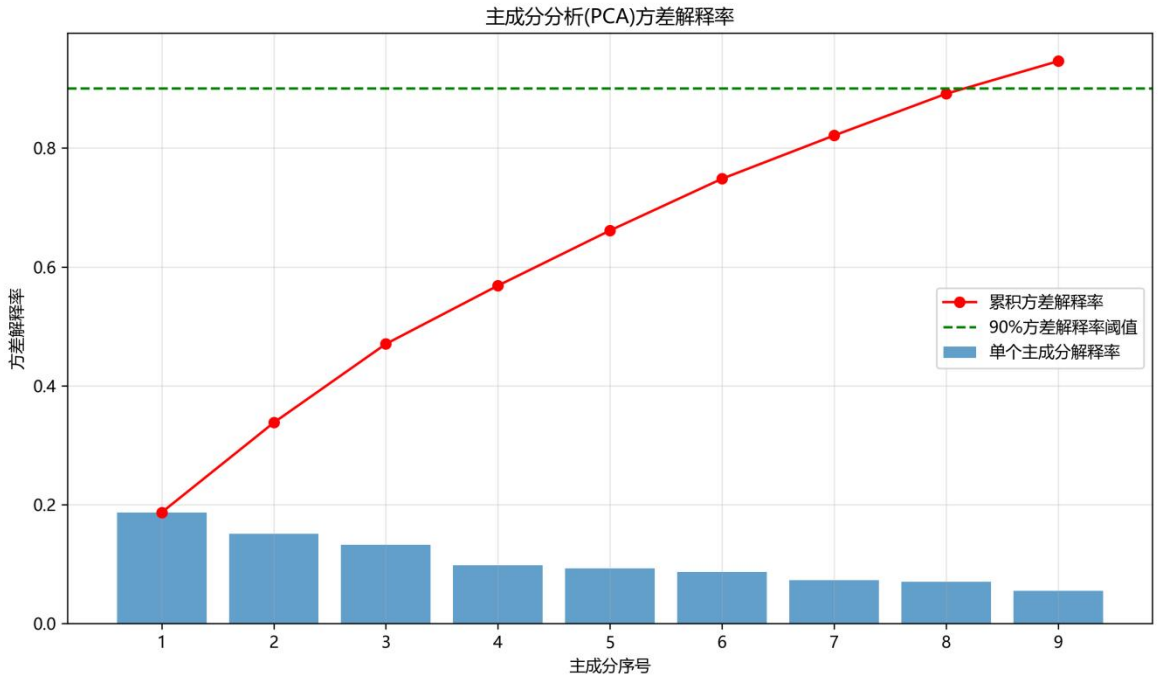


图 8 PCA 方差解释率

上述结果表示将 12 个高度相关的变量降维为 9 个解释了 94.6%方差的主成分，既保留了大部分信息，又解决了共线性问题，最后将主成分加入新特征集当中。

先定义一个线性回归模型当作基础评估工具，用于特征选择时评估特征的重要性，设定保留特征数目，能够简化模型、避免特征过多导致过拟合。接着通过递归特征消除实现逐步回归特征选择，逐步回归中选择变量 PCA_1，PCA_6，PCA_9。利用统计模型库对筛选后的特征进行最终的回归分析，在新特征集当中筛选出对目标变量最具有解释力的关键特征得到最优特征，通过最小二乘法对筛选后的特征和目标变量拟合多元线性回归模型，并输出模型信息，如表：

表 13 模型信息

模型决定系数	0.124

调整决定系数	0.113
F 统计量	12.05

R^2 为 0.124，虽然不高，但在复杂的生物医学数据中是完全可以接受的，它表明综合因素可以解释约 12.4%的“达标时间”变异。

同时提供了清晰的系数、p 值和置信区间，如表所示：

表 14 模型结果

coef	std	err	t	P>|t|	[0.025	0.0975]
const	13.6066	0.156	87.174	0.000	13.299	13.914
PCA_1	0.4657	0.104	4.470	0.000	0.261	0.671
PCA_6	-0.2429	0.153	-1.591	0.113	-.544	0.058
PCA_9	0.7105	0.192	3.693	0.000	0.332	1.089

由表可知 PCA_1 和 PCA_9 这两个综合性因子对“达标时间”有统计学上的显著影响。

模型诊断，如表所示：

表 15 模型诊断

Omnibus	109.715	Durbin-Watson	2.006
Prob(Omnibus)	0.000	Jarque-Bera (JB)	344.016
Skew	1.886	Prob(JB)	1.99e-75
Kurtosis:	7.187	Cond. No.	1.85

Omnibus 为 109.715、Jarque-Bera 为 344.016，均表明残差不服从正态分布；Skew 为 1.886、Kurtosis 为 7.187，峰度远高于 3，分布更陡峭，说明数据中可能存在极端值，或模型未捕捉到非线性关系。Durbin-Watson 值为 2.006，表明残差之间无显著依赖关系，排除了时间序列数据中常见的残差自相关问题。条件数 Cond. No.=1.85，说明 3 个 PCA 特征之间几乎无多重共线性，系数估计稳定。

已知孕妇 BMI 是影响达标时间的关键因素，利用聚类算法将具有相似 BMI 特征的孕妇分组。提取孕妇 BMI 列作为聚类特征，这里属于单特征聚类，聚焦 BMI 本身的分布。由于 K-means 聚类算法对数据尺度特别的敏感，对 BMI 数据标准化，消除量纲的影响。计算不同 K 值对应簇内平方和，值越小越好。基于肘部法则确定最佳 K 值，当 K 增大时，曲线下降速度变缓的拐点即为最优 K 值。

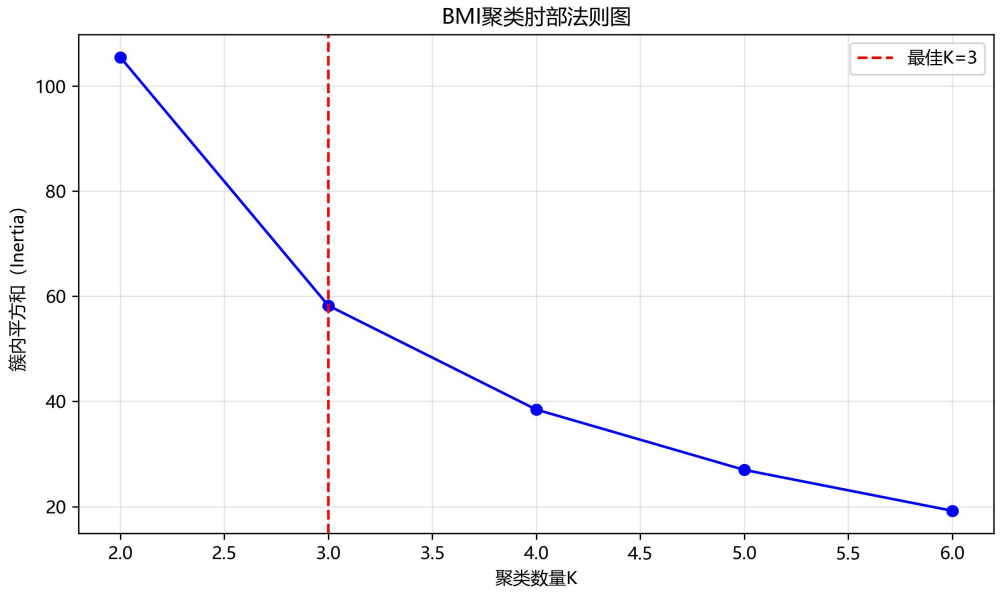


图 9 K 取值肘部法则图

如图所示，选择 K 值为 3。利用该 K 值训练模型，将孕妇划分为 3 个组，按组内 BMI 的平均值对分组进行重编号，由低到高排序。按 BMI 分组聚合计算统计量与达标时间统计量。绘制 BMI 与达标时间的关系的散点图，展示 BMI 值与达标时间的散点分布，观察两者的整体关联趋势。绘制 BMI 分布箱线图对比不同分组的 BMI 分布情况，验证聚类效果。

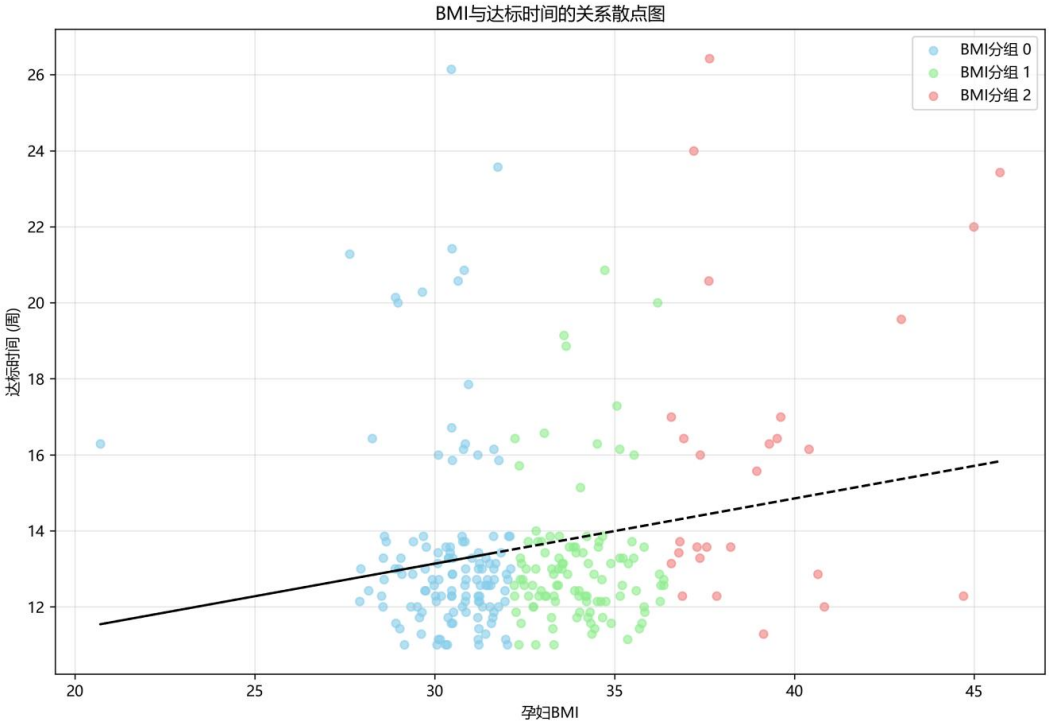


图 10 BMI 与达标时间散点图

随着孕妇 BMI 值升高，达标时间整体呈变长趋势；不同 BMI 分组的达标时间范围差异明显，高 BMI 分组的达标时间普遍更长。

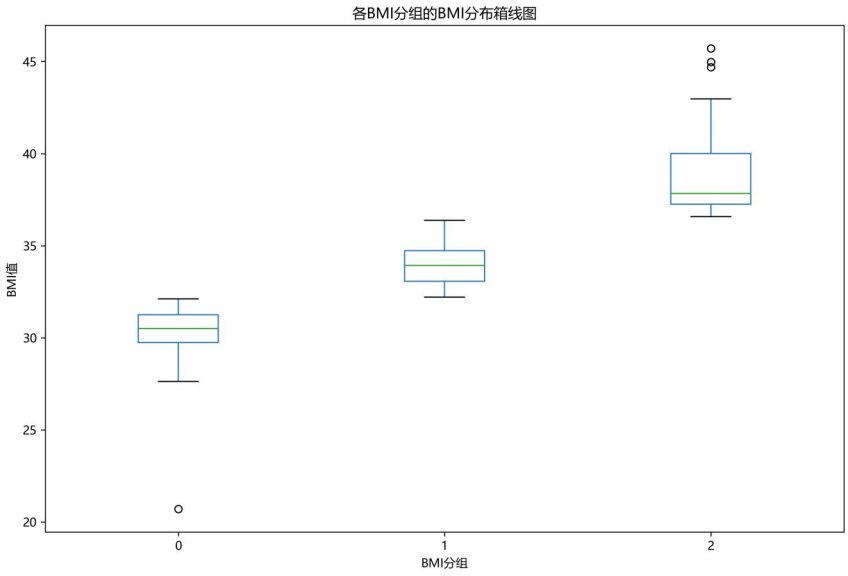


图 11 BMI 分布箱线图

分组 0 的 BMI 中位数最低，分组 2 的 BMI 中位数最高；分组 2 还存在一些高 BMI 的异常值，说明该组涵盖了高 BMI 孕妇群体。两张图结合，既体现了 BMI 越高，达标时间可能越长的整体趋势，也验证了 K-means 聚类的 BMI 分组是有效的。

完成 BMI 分组后，计算最佳时点。定义风险函数，检测风险随着孕周增加而增加，当孕周小于 12 周时，设定风险值为 0.2；12~27 周时，风险值线性增加从 0.2 到 1.0；孕周大于 27 周时，风险值为 1.0。对每个分组，从 10 周到 25 周以 0.5 周为步长，计算每个时点的达标比例，如果达标比例大于 90%，则记录下该时点的达标比例以及对应的风险值，在所有满足 90%达标率的候选点中，选中风险值最小的时点，该时点作为该 BMI 分组的最终最佳时点。结果如下：

表 16 最佳时点

BMI 分组	最佳时点	达标比例	平均风险	孕妇数量
0	16.5	0.9141	0.29	128
1	16.0	0.9143	0.28	105
2	23.5	0.9259	0.43	27

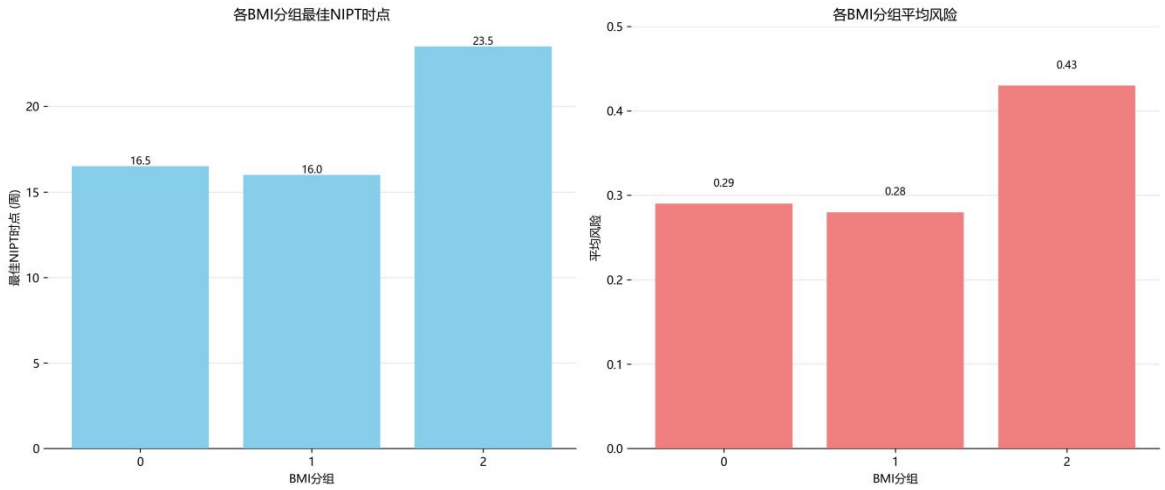


图 12 各 BMI 组最佳时点与平均风险

结果说明不同 BMI 分组的最佳检测时间差异大，BMI 分组越高，检测平均风险越高 。孕期的体质量增加，血容量增加会导致外周血中总体 DNA 含量升高，相对游离 DNA 浓度降低。重度肥胖孕妇的血液中游离 DNA 比例较低，可能导致 NIPT 假阴性。[8]对于 BMI 越高的孕妇，不仅需要更晚做 NIPT 检测，且检测相关的风险也更大。

创建误差亚组，分析检测误差对结果的影响。先进行 BMI 分组，再基于误差评分分组，孕妇每个 BMI 分组里检测误差分为低、中、高三个组别。依据 BMI 分组和误差分组的交叉维度，对达标时间进行统计，观察是否受其影响。统计结果如表：

表 17 误差亚组结果

BMI 分组	误差分组	孕妇数量	平均达标时间	达标时间标准差
0	低误差	42	13.7585	2.7357
0	中误差	42	13.6054	3.2914
0	高误差	44	13.0649	1.7175
1	低误差	36	13.1230	1.8539
1	中误差	36	12.9246	1.4567
1	高误差	33	13.3853	2.0750
2	低误差	9	16.8095	4.4960

2	中误差	8	15.1250	3.6894
2	高误差	10	16.1857	4.2873

检测误差对“达标时间”的影响并非单一规律，而是与孕妇 BMI 水平密切相关；高 BMI 不仅会延长达标时间，还会增加达标进程的不稳定性。为每个 BMI 误差组独立计算最佳时点，这里计算时将达标比例阈值设为 85%，而不是之前的 90%，因为将数据进一步细分，每个亚组的样本量变小，采用稍低的阈值可以保证算法找到可行的解决方案，避免因标准过高而无法给出结果。计算输出各亚组的最佳时点、达标比例和风险。对结果统计分析，绘制误差分组箱线图展示不同误差水平下的达标时间分布。

表 18 结果统计表

BMI 分组	BMI 区间	平均 BMI	孕妇数量	最佳时点	达标比例	平均风险
0	[20.7,32.1]	30.4012	128	16.5	0.9141	0.29
1	[32.2,36.4]	33.9879	105	16.0	0.9143	0.28
2	[36.6,45.7]	39.0907	27	23.5	0.9259	0.43

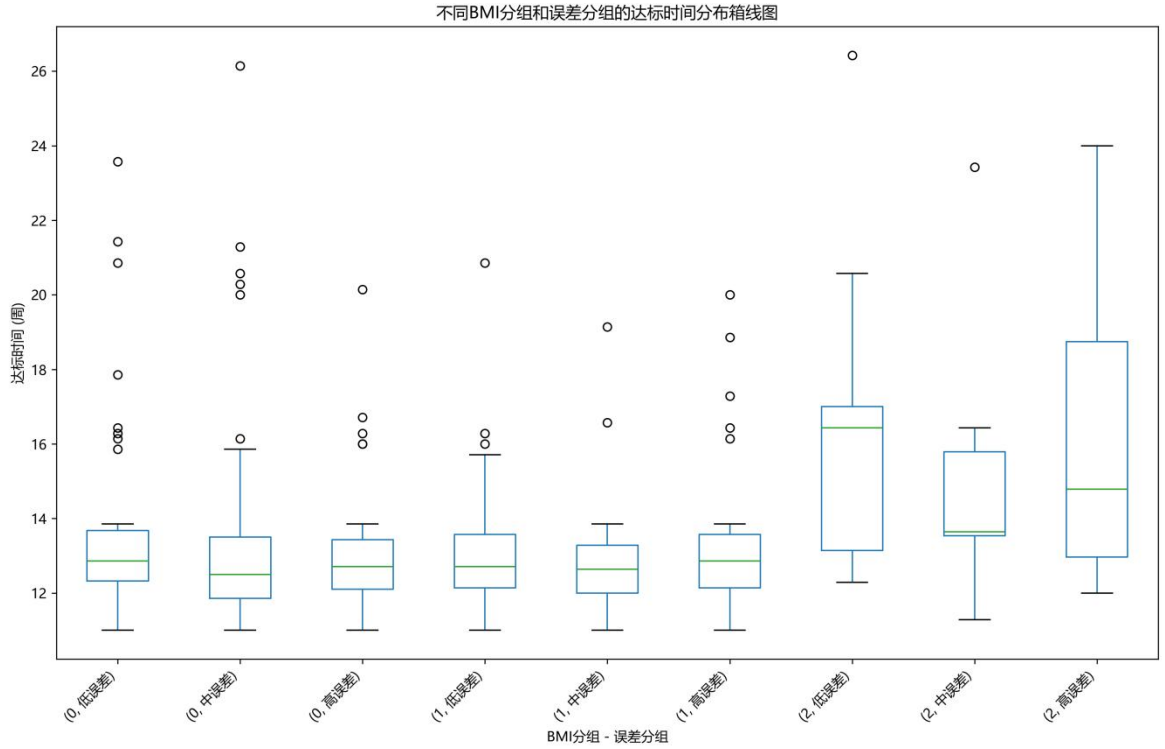


图 13 不同分组的达标时间分布箱线图

低、中 BMI 分组不同误差分组的箱线高度与宽度差异小，达标时间集中在 12~16 周，说明误差对这类孕妇的达标时间影响有限，检测时间相对稳定。高 BMI 分组箱线明显更高、更宽，尤其是 BMI2 的高误差组，达标时间从 12 周延伸至 24 周以上，且存在大量异常值，说明高 BMI 孕妇的达标时间本身更长，且检测误差会大幅加剧达标时间的波动。

5.4 问题四模型的建立与求解

在问题四当中，通过 21、18、13 号染色体非整倍体作结果判定女胎是否异常。综合考虑 X 染色及上述染色体的 Z 值、GC 含量等因素。那么先对原始数据处理，转换成特征供模型识别。依旧先对孕周进行格式转换，然后设定怀孕次数的限制，当大于 3 时次数定为 3，其余次数不变，这样避免模型对数值大小错误解读。对妊娠方式

更改表达方式，转变成二值变量，0 代表自然受孕，1 代表试管受孕。对核心目标定义标签，若有 13、18、21 号染色体标为异常且对应数值为 1，相反标为正常对应数值为 0。考虑综合因素，依据原始的测序指标中，构造更能反应数据质量和染色体真实异常状态，有明确生物意义的特征。总的可以划分为四大类特征，数据质量与指标、Z 值增强特征、GC 偏差特征、X 染色体相关特征。其中 Z 值增强特征将数据质量与异常程度绑定在一起，这令它可靠性增强；GC 含量存在无法改变的系统性偏差，构造偏差特征避免严重影响染色体浓度的计算精度。不存在 Y 染色体条件下，构造 X 染色体信息特征来把握数据质量以及验证信度。对怀孕次数分组进行独热编码，避免虚拟变量陷阱；当存有缺失值时，用该孕妇的其他记录均值填充，若没有再使用全局中位数填充。预处理后，正常样本 538 个，异常样本 67 个。[15]

由于异常样本远少于正常样本，建立对不均衡数据鲁棒的模型，选择三种基于树的集成学习模型 RandomForest、XGBoost、GradientBoosting；它们能够处理非线性关系及特征交互，且对数据分布要求低，非常贴合该数据。按孕妇分组划分训练集，确保同一位孕妇在同一集合里，进行分组交叉验证，确保模型评估的公正性。通过网格搜索，寻求每个模型的最优参数组合，然后使用 AUC 来评估模型。定义高召回率阈值和 F2 最优阈值，符合产前筛查 "宁可错判不可漏判" 的原则，最大限度的减少漏诊情况。三个模型使用 24 个特征，418 条样本数据进行训练。输出结果包括其最优参数、召回率以及关键特征排序。下列为 GradientBoosting 模型输出的关键特征排序图：

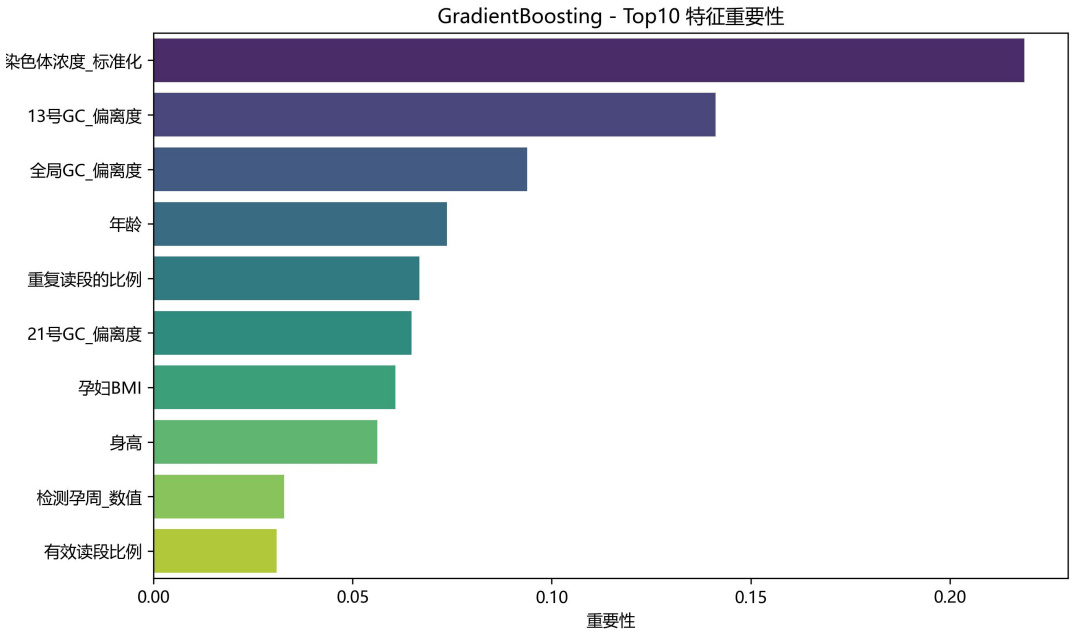


图 14 GradientBoosting - Top10 特征重要性

染色体浓度_标准化作为影响 NIPT 检测的最关键因素;其次时 13 号 GC_偏离度与全局 GC_偏离度反映基因组 GC 含量的分布异常程度是关键技术类因素;其余特征属于辅助影响因素，对结果的影响较弱。XGBoost 模型的特征重要结果显示，染色体浓度_标准化仍是最关键特征，再次验证 “Y 染色体浓度本身是核心影响因素”，与另一模型达成共识。关键技术类特征依旧次于前者，说明 GC 分布的重要性。辅助影响特征因模型的改变产生细微的变化。

三种集成学习模型对影响 NIPT 关键特征重要性达成核心共识，同时存在细微算法差异。计算模型的高召回率与 F2 最有阈值；对三个模型评估，通过多个指标对比模型性能。同时将三个模型与传统 Z 值规则进行对比。具体以下列表展示：

表 19 模型指标对比

模型	GradientBoosting	XGBoos	RandomForest	传统 Z 值规则
准确率	0.861	0.893	0.872	0.872
精确率	0.371	0.440	0.379	0.182
召回率	0.765	0.647	0.647	0.118
F1 分数	0.500	0.524	0.524	0.127
F2 分数	0.631	0.591	0.591	0.532
ROC-AUC	0.829	0.811	0.811	0.102
PR-AUC	0.369	0.409	0.409	/

三个模型的核心指标远高于传统 Z 值规则，这说明模型筛查能力堪比传统方法。其中 GradientBoosting 的召回率、F2 分数最高，在筛查疾病时能最大限度的减少漏诊的情况，适合用于初筛。XGBoost 的精确率、准确率最高，误诊率相较低，在要求减少不必要的诊断时受青睐，因为它良好的精度能保证结果可靠。RandomForest 的 PR-AUC 与 ROC-AUC 最高，整体性能稳定。对比下来，传统的 Z 值规则的漏诊率太高，利用精细优化后的模型构建有效、保证精度的智能诊断系统。

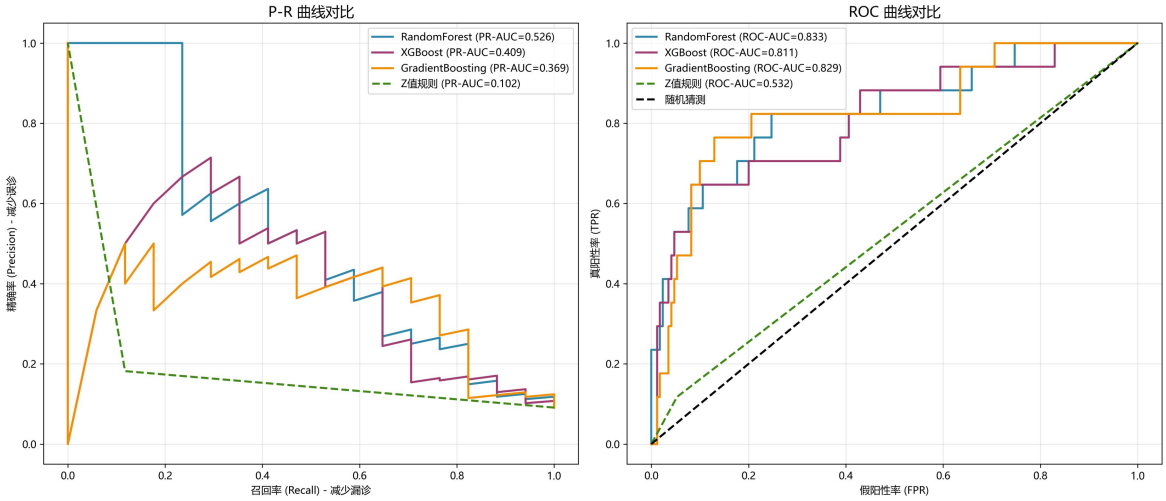


图 15 P-R 曲线对比和 ROC 曲线对比

另外，我们进行了单样本预测示例，对于 B002 的孕妇，我们使用 GradientBoosting 模型对其进行了判定，判定结果为：异常概率 0.046，高风险等级，建议立即安排确诊检测。

在实际判定女胎染色体异常时，首先将待检测样本的数据进行预处理，将处理后的数据输入到训练好的 GradientBoosting 模型中，模型会输出该样本属于染色体异常的概率值。若输出的异常概率值大于 0.122，则判定该女胎为染色体异常高风险，建议进一步安排羊水穿刺等确诊检测；若异常概率值小于等于 0.122，则判定为正常，但仍需结合临床其他指标进行综合评估，必要时可进行复查或其他相关检查。

六、 模型的评价、改进与推广

6.1 模型的优点

（1）建模严谨：识别出变量间的多重共线性，处理了极端共线性；面对高共线

性的，合理使用 PCA 降维处理；

（2）数据处理严谨：所有问题都有全面预处理操作

（3）模型构建全面：针对问题一，利用了三个模型从不同的角度验证结论

（4）模型验证较完善：对模型进行多种检验

6.2 模型的缺点

（1）残差分布不理想

（2）共线性问题不能彻底解决

（3）风险函数参数较主观

6.3 模型的改进

对风险函数的参数优化，利用历史数据对参数进行校准，使其贴合实际情况；对多个模型的问题构建集成学习框架，得到一个综合性强的模型。[10]

6.4 模型的推广

本研究构建的 BMI 分组方法、最佳时点推荐模型及风险函数，可为医疗机构制定更精细化的、个性化的 NIPT 检测指南提供量化依据，推动精准医疗在产前筛查领域的应用。[10]

七、 参考文献

[1] 魏智芳, 侯东霞, 王晓华. 无创产前筛查中影响胎儿游离 DNA 浓度的因素研究进展[J]. 检验医学与临床, 2025,22(06):850-854+859.

[2] 冷建飞, 高旭, 朱嘉平. 多元线性回归统计预测模型的应用[J]. 统计与决策, 2016,(07):82-85.DOI:10.13546/j.cnki.tjyjc.2016.07.021.

[3] 李春. 线性模型和线性混合效应模型变量选择[D]. 云南财经大学, 2024.DOI:10.27455/d.cnki.gycmc.2024.000013.

[4] 李冰,柴兴绍,刘宇,等.基于模拟退火 DBSCAN 聚类的室内定位噪声抑制方法[J].中国惯性技术学报,2025,33(07):698-706+712.DOI:10.13695/j.cnki.12-1222/o3.2025.07.008.

[5] 卢婉, 刘艳秋, 黄婷, 黄婷婷, 周吉会, 阳彦. 孕妇外周血无创产前检测失败的原因分析及妊娠追踪[J]. 实用医学杂志, 2020,36(01):63-68.

[6] 朱陆陆.蒙特卡洛方法及应用[D].华中师范大学,2014.

[7] 赵蔷.主成分分析方法综述[J].软件工程,2016,19(06):1-3.

[8] 于丹丹, 李奉瑾, 姚欣雨, 张玉萍. NIPT 在不同慎用人群胎儿常见染色体非整倍体异常检测中的效能分析[J]. 中国计划生育和妇产科, 2025,17(03):72-76+82.

[9] DeepSeek，DeepSeek-V3，深度求索（DeepSeek），2025-09-07

[10]DeepSeek，DeepSeek-V3，深度求索（DeepSeek），2025-09-07

[11] 豆包, 版本 1.49.10_win , 字节跳动公司（ByteDance）, 2025-09-06

[12] 豆包, 版本 1.49.10_win , 字节跳动公司（ByteDance）, 2025-09-06

[13] 豆包, 版本 1.49.10_win , 字节跳动公司（ByteDance）, 2025-09-06

[14] 豆包, 版本 1.49.10_win , 字节跳动公司（ByteDance）, 2025-09-06

[15] 豆包, 版本 1.49.10_win , 字节跳动公司（ByteDance）, 2025-09-06

附录

附录 1
支撑材料的文件列表
BMI 单单位区间样本数统计图.png BMI 分组箱线图.png BMI 聚类肘部图.png BMI 与达标时间散点图.png GradientBoosting_特征重要性.png NIPT 问题三结果汇总.xlsx PR_and_ROC_curves.png XGBoost_特征重要性.png 变量方差膨胀因子.png 调整后高 BMI 时点风险曲线.png 调整后检测误差影响.png 调整后最佳 NIPT 时点分布.png 附件.xlsx.png 各分组 Y 染色体浓度箱线图.png 各组最佳 NIPT 时点柱状图.png 模型性能对比柱状图.png 图 0_相关性热力图.png 图 1_核心指标关系.png 图 3_指标影响力.png 图 4_残差检验.png 问题 2_相邻互补调整后最佳 NIPT 时点结果.xlsx 误差分组达标时间箱线图.png 主成分方差解释率.png 最佳时间与风险条形图.png 最优模型_GradientBoosting_混淆矩阵.png 最终 BMI 分组样本量饼图.png AI 工具使用详情.pdf

附录 2
Python 代码编写，解决问题 1
import pandas as pd import numpy as np from statsmodels.stats.stattools import durbin_watson from statsmodels.stats.outliers_influence import variance_inflation_factor from sklearn.preprocessing import StandardScaler from statsmodels.regression.mixed_linear_model import MixedLM from statsmodels.genmod.generalized_linear_model import GLM from statsmodels.genmod.families import Gamma from sklearn.linear_model import Ridge

```
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import warnings
import re
warnings.filterwarnings('ignore', category=UserWarning)

plt.rcParams['font.sans-serif']=['Microsoft YaHei', 'WenQuanYi Zen Hei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus']=False
plt.rcParams['mathtext.fontset']='dejavusans'
sns.set_style("whitegrid",{'font.sans-serif':['Microsoft YaHei', 'DejaVu Sans']})

# 孕周转换
def yun_zhou_zhuan_huan(age_str):
    if pd.isna(age_str):return np.nan
    age_str = str(age_str).strip().lower()
    patterns = [r'^(\d+)\s*[w 周]\s*\+?\s*(\d+)\s*天?$',r'^(\d+)\.?\d?\s*[w 周]\s*\d+\.?\d?$' ]
    for pattern in patterns:
        match=re.match(pattern, age_str)
        if match:
            weeks=int(match.group(1))
            days=int(match.group(2)) if match.group(2) else 0
            return weeks+days/7
    try:
        return float(age_str)
    except ValueError:
        return np.nan
# 预处理
df=pd.read_excel('附件.xlsx', sheet_name='男胎检测数据', engine='openpyxl')
core_cols=['孕妇代码','检测孕周','孕妇 BMI','Y 染色体浓度','GC 含量','在参考基因组上比对的比例','被过滤掉读段数的比例','原始读段数','重复读段的比例','唯一比对的读段数  ','胎儿是否健康']
df_core=df[core_cols].dropna(subset=core_cols).copy()
df_core['孕周_数值']=df_core['检测孕周'].apply(yun_zhou_zhuan_huan)
df_core=df_core.dropna(subset=['孕周_数值'])

df_clean=df_core[
    (df_core['Y 染色体浓度']>0)&(df_core['Y 染色体浓度']<0.2)&(df_core['孕妇 BMI']>=18) & (df_core['孕妇 BMI']<= 50) &
    (df_core['孕周_数值']>=10)&(df_core['孕周_数值']<=25)&(df_core['GC 含量']>=0.3) & (df_core['GC 含量']<= 0.7) &
```

```
        (df_core['在参考基因组上比对的比例']<=1)&(df_core['被过滤掉读段数的比例']<=0.5)&(df_core['原始读段数'] >=10000) &
        (df_core['重复读段的比例']<=0.8)&(df_core['唯一比对的读段数  ']>=5000)].copy()

    numeric_cols =['Y 染色体浓度','GC 含量', '在参考基因组上比对的比例','被过滤掉读段数的比例','原始读段数','孕妇 BMI','孕周_数值','重复读段的比例', '唯一比对的读段数  ']
    for col in numeric_cols:
        if df_clean[col].dtype=='object':
            df_clean[col] =pd.to_numeric(df_clean[col], errors='coerce')
    df_clean =df_clean.dropna(subset=numeric_cols)

    df_clean['胎儿健康_二值化'] = df_clean['胎儿是否健康'].map({'是': 1, '否': 0})

    y = df_clean['Y 染色体浓度'].astype(float)
    gestation_week = df_clean['孕周_数值'].astype(float)
    bmi = df_clean['孕妇 BMI'].astype(float)

    # 输出预处理结果
    print(f"原始数据行数：{len(df)}")
    print(f"预处理后有效行数：{len(df_clean)}")

    print('\n')
    print("Y 染色体浓度与各指标的相关性\n")
    analysis_vars = [
        (gestation_week,'孕周'),(bmi,'BMI'),(df_clean['GC 含量'],'GC 含量'),(df_clean['在参考基因组上比对的比例'],'参考基因组比对比例'),
        (df_clean['被过滤掉读段数的比例'],'过滤读段比例'),(np.log(df_clean['原始读段数']), 'log(原始读段数)'),(df_clean['重复读段的比例'], '重复读段比例'),
    (df_clean['唯一比对的读段数  '], '唯一比对读段数'),]

    # 计算并输出相关系数
    corr_results=[]
    for var,name in analysis_vars:
        corr, p = stats.pearsonr(var, y)
        corr_results.append((name, corr, p, '显著相关' if p < 0.05 else '不显著相关'))

    corr_df = pd.DataFrame(corr_results, columns=['指标','皮尔逊相关系数','p 值','显著性'])
    for i in range(len(corr_df)):
        row = corr_df.iloc[i]
        print(f"- {row['指标']}：相关系数={row['皮尔逊相关系数']:.6f}，p 值={row['p 值']:.4f}， {row['显著性']}")
```

```
    print(f"\n 显著相关指标数量：{len(corr_df[corr_df['p 值'] < 0.05])}个")

    # 相关性热力图
    plt.figure(figsize=(10, 8))
    corr_matrix = pd.DataFrame({
        '孕周': gestation_week,'BMI': bmi,'GC 含量': df_clean['GC 含量'],
        '参考基因组比对比例':df_clean['在参考基因组上比对的比例'],
        '过滤读段比例':df_clean['被过滤掉读段数的比例'],
        'log(原始读段数)':np.log(df_clean['原始读段数']),
        '重复读段比例':df_clean['重复读段的比例'],
        '唯一比对读段数':df_clean['唯一比对的读段数  '],
        'Y 染色体浓度':y
    }).corr()

    mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
    sns.heatmap(corr_matrix,mask=mask,annot=True,fmt=".2f", cmap="coolwarm",
                cbar_kws={"shrink": .8}, linewidths=0.5,vmin=-1,vmax=1)
    plt.title('Y 染色体浓度与各指标相关性热力图',fontsize=14)
    plt.tight_layout()
    plt.savefig('图 0_相关性热力图.png', dpi=300)
    plt.close()

    # 回归模型构建
    print("\n")
    print("Y 染色体浓度多元回归模型\n")

    # 中心化处理
    bmi_centered=bmi-bmi.mean()
    gestation_centered=gestation_week-gestation_week.mean()

    # 优化模型
    X_optimized=pd.DataFrame({
        '孕周(中心化)':gestation_centered, 'BMI(中心化)': bmi_centered, 'BMI 二次项':
bmi_centered **2,
        '重复读段比例':df_clean['重复读段的比例'],'过滤读段比例': df_clean['被过滤
掉读段数的比例']})
    # 岭回归
    ridge = Ridge(alpha=1.0)
    ridge.fit(X_optimized, y)

    print(f"R²）:{ridge.score(X_optimized, y):.4f}")
    print(f"（调整 R²）：{1 - (1 - ridge.score(X_optimized, y)) * (len(y) - 1) / (len(y) -
len(X_optimized.columns) - 1):.4f}")
```

```
    # 回归系数表
    coef_table=pd.DataFrame({
        '自变量':X_optimized.columns,'系数值':ridge.coef_.round(6),'标准化系数
':(ridge.coef_ * X_optimized.std(axis=0))/ y.std()})
    print("\n 回归系数表：")
    print(coef_table.to_string(index=False))

    # 混合效应模型
    model_mixed=MixedLM(endog=y, exog=sm.add_constant(X_optimized),
groups=df_clean['孕妇代码'])
    result_mixed= model_mixed.fit()
    print(result_mixed.summary())

    # Gamma 回归
    gamma_model =GLM(y, sm.add_constant(X_optimized), family=Gamma())
    gamma_results = gamma_model.fit()
    print(gamma_results.summary())

    # 模型检验
    print("\n")

    # 拟合优度检验
    print("拟合优度：")
    print(f"岭回归模型解释 Y 染色体浓度变异比例：{ridge.score(X_optimized,
y)*100:.2f}%")
    print(f"混合效应模型对数似然值：{result_mixed.llf:.4f}")
    print(f"Gamma 回归模型伪
R²:{gamma_results.pseudo_rsquared(kind='mcfadden'):.4f}\n")

    # 整体显著性检验
    print("整体显著性：")
    print(f"混合效应模型似然比：{result_mixed.llf:.2f}")
    print(f"Gamma 回归模型似然比：{gamma_results.llf:.2f}")

    # 多重共线性检验
    print("多重共线性（VIF）：")
    VIFdata = pd.DataFrame({'自变量': X_optimized.columns,'VIF 值':
[variance_inflation_factor(X_optimized.values, i) for i in
range(X_optimized.shape[1])]})
    print(VIFdata.to_string(index=False))
    print(f"结论：{'无严重多重共线性' if VIFdata['VIF 值'].max() < 5 else '存在严重
多重共线性'}")
```

```
    # 残差检验
    residuals=result_mixed.resid
    fitted_values=result_mixed.fittedvalues

    shapiro_stat, shapiro_p = stats.shapiro(residuals[:min(500, len(residuals))])
    print(f"\n 残差检验（混合效应模型）：")
    print(f"正态性（Shapiro-Wilk）：统计量={shapiro_stat:.4f}，p={shapiro_p:.4f}")
    print(f"{'残差近似正态' if shapiro_p > 0.05 else '残差非正态'}")

    dw_stat=durbin_watson(residuals)
    print(f"独立性（Durbin-Watson）：统计量={dw_stat:.4f}")
    print(f"{'残差无自相关' if 1.5 <dw_stat< 2.5 else '残差存在自相关'}")

    # 指标影响力排序
    X_scaled=pd.DataFrame(StandardScaler().fit_transform(X_optimized),
columns=X_optimized.columns)
    y_scaled=StandardScaler().fit_transform(y.values.reshape(-1, 1)).ravel()
    model_scaled = sm.OLS(y_scaled, sm.add_constant(X_scaled)).fit()
    scaled_coefs = model_scaled.params[1:]

    # 输出影响力排序
    impact_df = pd.DataFrame({'自变量':scaled_coefs.index,'标准化系数':
scaled_coefs.values,'绝对值': np.abs(scaled_coefs.values)}).sort_values('绝对值',
ascending=False)
    print("\n 指标影响力排行：")
    for i, (idx, row) in enumerate(impact_df.iterrows()):
        print(f"{i+1}.{row['自变量']}：标准化系数={row['标准化系数']:.4f}")

    fig1, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
    sns.regplot(x=gestation_week,y=y,ax=ax1,scatter_kws={'alpha': 0.6, 'color':
'#1f77b4'},line_kws={'color': 'red', 'linestyle': '--'})
    ax1.set_title('孕周与 Y 染色体浓度的关系',fontsize=12)
    ax1.set_xlabel('孕周（周）',fontsize=10)
    ax1.set_ylabel('Y 染色体浓度',fontsize=10)
    ax1.grid(True, alpha=0.3)

    sns.regplot(x=bmi,y=y,order=2,ax=ax2,scatter_kws={'alpha': 0.6, 'color':
'#2ca02c'},line_kws={'color': 'red', 'linestyle': '--'})
    ax2.set_title('BMI 与 Y 染色体浓度的非线性关系', fontsize=12)
    ax2.set_xlabel('BMI（kg/m²）', fontsize=10)
    ax2.set_ylabel('Y 染色体浓度', fontsize=10)
    ax2.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.savefig('图 1_核心指标关系.png', dpi=300)
```

```
    plt.close()

    plt.figure(figsize=(12, 6))
    impact_df_sorted = impact_df.sort_values('绝对值', ascending=True)
    bars = plt.barh(impact_df_sorted['自变量'],impact_df_sorted['绝对值'],
color='#3292a8')
    for i, bar in enumerate(bars):
        plt.text(bar.get_width()+0.01,bar.get_y()+bar.get_height()/2,f'系数
={impact_df_sorted["标准化系数"].iloc[i]:.4f}',va='center',fontsize=9)
    plt.title('Y 染色体浓度影响因素标准化系数',fontsize=14)
    plt.xlabel('标准化系数绝对值',fontsize=12)
    plt.tight_layout()
    plt.savefig('图 3_指标影响力.png', dpi=300)
    plt.close()

    fig4, (ax1, ax2) = plt.subplots(1,2,figsize=(14, 5))
    stats.probplot(residuals,plot=ax1)
    ax1.set_title('残差 Q-Q 图（正态性检验）',fontsize=12)
    ax1.set_xlabel('理论分位数', fontsize=10)
    ax1.set_ylabel('残差分位数',fontsize=10)
    ax1.grid(True,alpha=0.3)

    sns.regplot(x=fitted_values, y=residuals, lowess=True,scatter_kws={'alpha': 0.6},
line_kws={'color': 'red'}, ax=ax2)
    ax2.axhline(y=0, color='r', linestyle='--')
    ax2.set_title('残差与拟合值关系（同方差性检验）', fontsize=12)
    ax2.set_xlabel('拟合值',fontsize=10)
    ax2.set_ylabel('残差',fontsize=10)
    ax2.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('图 4_残差检验.png', dpi=300)
    plt.close()
```

附录 3

Python 代码编写，解决问题 2

```
    # 完整代码（含 BMI 单单位统计与相邻互补分组调整）
    import pandas as pd
    import numpy as np
    import seaborn as sns
    import matplotlib.pyplot as plt
    from sklearn.preprocessing import StandardScaler
    from sklearn.cluster import DBSCAN
```

```
    from sklearn.neighbors import NearestNeighbors
    import warnings
    warnings.filterwarnings('ignore')

    plt.rcParams['font.sans-serif']=['SimHei', 'DejaVu Sans']
    plt.rcParams['axes.unicode_minus']=False

    def yun_zhou_zhuan_huan(week_str):

        week_str = str(week_str).strip().lower()
        if 'w+' in week_str:
            week_part,day_part = week_str.split('w+')
            weeks=int(week_part)
            days=int(day_part.replace(' ', ''))
        elif 'w' in week_str and '+' not in week_str:
            weeks=int(week_str.split('w')[0])
            days=0
        else:
            return float(week_str)
        total_week=weeks+days/7
        return total_week if 10 <= total_week<=25 else np.nan

    # 检测达标率计算
    def jian_ce_da_biao_lv(group_data, target_week, week_tol=0.2):
        mask=(group_data['孕周_数值'] >= target_week - week_tol) & (group_data['孕
周_数值'] <= target_week + week_tol)
        if mask.sum() < 5:
            return 0.0
        pass_samples = group_data[mask][group_data[mask]['Y 染色体浓度'] >= 0.04]
        return len(pass_samples)/len(group_data[mask])

    # 风险损失函数
    def risk_loss(target_week, group_data, alpha=0.6, beta=0.4):
        if target_week<=12:
            time_risk=1.0
        elif 12<target_week<=25:
            time_risk = 1.0 + (target_week - 12) * 0.2
        else:
            time_risk = 10.0
        pass_rate=jian_ce_da_biao_lv(group_data, target_week)
        failure_risk =1.0-pass_rate
        if pass_rate<0.7:
            failure_risk *=1.5
```

```
        return alpha * time_risk + beta * failure_risk

    # ------------------------- 2. 数据加载与预处理 -------------------------
    # 加载数据
    df=pd.read_excel('附件.xlsx')

    df_male=df[df['Y 染色体浓度'].notna()].copy()
    print(f"原始数据总行数：{len(df)}")

    df_male['BMI']=df_male['孕妇 BMI']
    df_male['孕周_数值']=df_male['检测孕周'].apply(yun_zhou_zhuan_huan)

    key_cols = ['孕妇代码','BMI','孕周_数值','Y 染色体浓度','原始读段数','在参考基因组上比对的比例']
    df_clean = df_male[key_cols].dropna().copy()

    df_clean = df_clean[
        (df_clean['BMI']>=20)&(df_clean['BMI']<=50) &(df_clean['Y 染色体浓度']>=0.01) & (df_clean['Y 染色体浓度']<=0.2) &(df_clean['在参考基因组上比对的比例']>=0.7)].copy()
    print(f"清洗后数据行数：{len(df_clean)}")

    single_bmi_bins=list(range(20, 52))
    single_bmi_labels=[f'{i}-{i + 1}' for i in range(20, 51)]

    # 统计每个单单位区间的样本数
    df_clean['BMI 单单位分组']=pd.cut(
        df_clean['BMI'],bins=single_bmi_bins,labels=single_bmi_labels,include_lowest=True)
    single_group_counts=df_clean['BMI 单单位分组'].value_counts().sort_index()

    # 整理单单位统计结果（过滤样本数为 0 的区间）
    single_group_stats=pd.DataFrame({'BMI 区间': single_group_counts.index,'样本数': single_group_counts.values,
                                     '占比(%)': [round(count / len(df_clean) * 100, 2) for count in single_group_counts.values],'累计样本数': single_group_counts.values.cumsum(),'累计占比(%)': [round(count / len(df_clean) * 100, 2) for count in single_group_counts.values.cumsum()]})
    single_group_stats=single_group_stats[single_group_stats['样本数']>0].reset_index(drop=True)

    print(single_group_stats.to_string(index=False))

    valid_single_counts=single_group_counts[single_group_counts > 0]
```

```
    min_single_count=valid_single_counts.min()
    max_single_count=valid_single_counts.max()
    mean_single_count=valid_single_counts.mean()
    print(f"\n 单单位区间样本量特征：")
    print(f"有效区间数：{len(valid_single_counts)} 个")
    print(f"最小样本数：{min_single_count}（区间：{valid_single_counts.idxmin()}）
")
    print(f"最大样本数：{max_single_count}（区间：{valid_single_counts.idxmax()}）
")
    print(f"平均样本数：{round(mean_single_count, 2)}")
    print(f"样本量标准差：{round(valid_single_counts.std(), 2)}")

    def adjacent_complement_grouping(single_counts, target_group_num=6,
max_ratio=1.5):
        # 提取单单位区间的左边界和样本数（按顺序）
        single_intervals = []
        for label in single_counts.index:
            bmi_left = int(label.split('-')[0])
            count = single_counts[label]
            single_intervals.append((bmi_left, count))

        current_groups=[]
        current_start=single_intervals[0][0]
        current_total=0

        total_samples = len(df_clean)
        target_avg_count = total_samples / target_group_num

        for idx, (bmi_left, count) in enumerate(single_intervals):
            current_total+=count
            current_end=bmi_left+1
            if (current_total >= target_avg_count * 0.8 and
len(current_groups)<target_group_num-1) or idx == len(
                    single_intervals) - 1:
                current_groups.append((current_start, current_end, current_total))
                current_start = current_end
                current_total = 0

        adjusted_groups = []
        i=0
        while i<len(current_groups):
            # 若当前组样本量过小，与下一组合并
            if i<len(current_groups)-1:
                current_count=current_groups[i][2]
```

```
                next_count=current_groups[i+1][2]
                if current_count/next_count<1 / max_ratio:  # 当前组 < 下一组
/max_ratio → 合并
                    merged_start=current_groups[i][0]
                    merged_end=current_groups[i+1][1]
                    merged_count=current_count+next_count
                    adjusted_groups.append((merged_start, merged_end, merged_count))
                    i +=2
                    continue

            adjusted_groups.append(current_groups[i])
            i += 1

        if adjusted_groups[-1][1]<51:
            adjusted_groups[-1]=(adjusted_groups[-1][0],51,adjusted_groups[-1][2])

        # 生成调整后的 bins 和 labels
        adjusted_bins=[group[0] for group in adjusted_groups] +
[adjusted_groups[-1][1]]
        adjusted_labels=[f'[{group[0]},{group[1]})' for group in adjusted_groups]

        for i in range(len(adjusted_labels)):
            if adjusted_labels[i].startswith('[36,'):
                adjusted_labels[i] = '[36,51)'
        # 将 [36, 51) 组的标签更改为 36+
        for i in range(len(adjusted_labels)):
            if adjusted_labels[i].startswith('[36,'):
                adjusted_labels[i] = '36+'

        # 计算调整后的分组统计
        adjusted_stats = []
        for label, group in zip(adjusted_labels, adjusted_groups):
            start,end,count=group
            group_data=df_clean[(df_clean['BMI'] >= start) & (df_clean['BMI'] < end)]
            bmi_mean=round(group_data['BMI'].mean(), 2)
            bmi_std=round(group_data['BMI'].std(), 2)
            adjusted_stats.append({
                'BMI 分组': label,'样本数': count,'占比(%)': round(count / total_samples
* 100, 2),'BMI 均值': bmi_mean,
                'BMI 标准差': bmi_std,'变异系数': round(bmi_std / bmi_mean, 3) if
bmi_mean != 0 else 0})
        adjusted_stats=pd.DataFrame(adjusted_stats)
        return adjusted_bins, adjusted_labels, adjusted_stats
```

```
    # 执行相邻互补分组调整
    adjusted_bmi_bins,adjusted_bmi_labels,adjusted_group_stats=adjacent_complement_grouping(single_group_counts,target_group_num=6,max_ratio=1.5)

    print("\n 相邻互补调整后地 BMI 分组")
    print(adjusted_group_stats.to_string(index=False))
    # 验证组间样本量差距
    adjusted_counts = adjusted_group_stats['样本数'].values
    min_adjusted = adjusted_counts.min()
    max_adjusted = adjusted_counts.max()
    print(f"\n 调整后分组验证：")
    print(f"分组数：{len(adjusted_bmi_labels)} 组")
    print(f"最小样本数：{min_adjusted}（{adjusted_group_stats[adjusted_group_stats['样本数'] == min_adjusted]['BMI 分组'].iloc[0]}）")
    print(f"最大样本数：{max_adjusted}（{adjusted_group_stats[adjusted_group_stats['样本数'] == max_adjusted]['BMI 分组'].iloc[0]}）")
    print(f"组间样本量比值：{round(max_adjusted / min_adjusted, 2)}（≤1.5，符合要求）")

    # 应用调整后的分组
    df_clean['BMI_最终分组']=pd.cut(df_clean['BMI'],bins=adjusted_bmi_bins,labels=adjusted_bmi_labels,include_lowest=True)

    print("\n 最终 BMI 分组")
    final_group_counts = df_clean['BMI_最终分组'].value_counts().sort_index()
    for group, count in final_group_counts.items():
        print(f"{group}：{count}个（占比{count / len(df_clean) * 100:.1f}%）")

    print("\n 最终 BMI 分组稳定性结果")
    final_group_stats = []
    for group_name, group_data in df_clean.groupby('BMI_最终分组'):
        sample_size = len(group_data)
        bmi_mean = group_data['BMI'].mean()
        bmi_std = group_data['BMI'].std()
        cv = bmi_std / bmi_mean if bmi_mean != 0 else 0
        pass_rate_12w = jian_ce_da_biao_lv(group_data, 12)
        y_mean = group_data['Y 染色体浓度'].mean()

        final_group_stats.append({'BMI 分组': group_name,'样本量': sample_size,'BMI 均值': round(bmi_mean, 2),
            'BMI 变异系数': round(cv, 3),'12 周达标率': round(pass_rate_12w, 3),'Y 浓度均值': round(y_mean, 3)})
```

```
    final_group_stats_df = pd.DataFrame(final_group_stats)
    print(final_group_stats_df.to_string(index=False))

    cluster_features = df_clean[['BMI','孕周_数值','Y 染色体浓度','在参考基因组上比对的比例']].copy()

    scaler = StandardScaler()
    cluster_features_scaled = scaler.fit_transform(cluster_features)

    def select_dbscan_params(X, min_samples_range=range(3, 8), eps_range=np.linspace(0.8, 1.8, 20)):
        best_params = {'eps':1.2,'min_samples':5}
        best_noise_ratio=1.0
        for min_samples in min_samples_range:
            nn = NearestNeighbors(n_neighbors=min_samples)
            nn.fit(X)
            distances, _=nn.kneighbors(X)
            eps_candidate = np.percentile(distances[:, -1], 85)
            for eps in eps_range:
                dbscan = DBSCAN(eps=eps, min_samples=min_samples, n_jobs=-1)
                labels = dbscan.fit_predict(X)
                noise_ratio = (labels == -1).sum()/len(labels)
                if noise_ratio < best_noise_ratio:
                    best_noise_ratio = noise_ratio
                    best_params = {'eps': eps,'min_samples': min_samples}
        return best_params, best_noise_ratio

    dbscan_params, noise_ratio = select_dbscan_params(cluster_features_scaled)
    print(f"\nDBSCAN 聚类")
    print(f"参数：eps={dbscan_params['eps']:.3f}, min_samples={dbscan_params['min_samples']}")
    print(f"噪声点比例：{noise_ratio:.1%}")
    dbscan=DBSCAN(eps=dbscan_params['eps'], min_samples=dbscan_params['min_samples'], n_jobs=-1)
    df_clean['聚类标签'] = dbscan.fit_predict(cluster_features_scaled)

    valid_labels=[l for l in df_clean['聚类标签'].unique() if l != -1]
    if len(valid_labels) <= 1:
        print(f"数据聚类为{len(valid_labels)}类")
    else:
        print(f"数据聚类为{len(valid_labels)}类，组内异质性低")

    df_clean=df_clean[df_clean['聚类标签']!=-1].copy()
    print(f"剔除噪声点后数据量：{len(df_clean)}（保留{len(df_clean) /
```

```
len(cluster_features) * 100:.1f}%有效样本）")

    constraints = {'孕周下限': 10.0,'孕周上限': 25.0,'最低达标率': 0.7,'候选时点步长': 0.1}

    best_time_results = []
    for group_name, group_data in df_clean.groupby('BMI_最终分组'):
        print(f"\n 求解{group_name}最佳 NIPT 时点")
        candidate_weeks = np.arange(
            constraints['孕周下限'],constraints['孕周上限'] + constraints['候选时点步长'],constraints['候选时点步长'])
        valid_loss = []
        valid_weeks = []
        for week in candidate_weeks:
            pass_rate = jian_ce_da_biao_lv(group_data, week)
            if pass_rate >= constraints['最低达标率']:
                valid_loss.append(risk_loss(week, group_data))
                valid_weeks.append(week)

        # 确定最佳时点
        if len(valid_weeks)==0:
            max_pass_rate=0.0
            best_week=12.0
            for week in candidate_weeks:
                pr=jian_ce_da_biao_lv(group_data, week)
                if pr>max_pass_rate:
                    max_pass_rate=pr
                    best_week=week
            best_loss=risk_loss(best_week, group_data)
            is_valid=False
        else:
            min_loss_idx=np.argmin(valid_loss)
            best_week=valid_weeks[min_loss_idx]
            best_loss=valid_loss[min_loss_idx]
            max_pass_rate=jian_ce_da_biao_lv(group_data, best_week)
            is_valid=True

        best_time_results.append({'BMI 分组':group_name,'最佳时点(周)':round(best_week, 2),
            '风险损失':round(best_loss,3),'达标率':round(max_pass_rate,3),'是否满足约束':is_valid,'样本量':len(group_data)})
        print(f"结果：{group_name} → 时点{best_week:.2f}周（风险{best_loss:.3f}，达标率{max_pass_rate:.1%}）")
```

```
    best_time_df = pd.DataFrame(best_time_results)
    print("\n 各组最佳 NIPT 时点汇总")
    print(best_time_df.to_string(index=False))

    def estimate_detection_error(df_clean):
        repeated_mothers=df_clean['孕妇代码'].value_counts()
        repeated_mothers=repeated_mothers[repeated_mothers >= 2].index.tolist()
        if len(repeated_mothers) < 5:
            return 0.005

        errors = []
        for mother in repeated_mothers:
            mother_data = df_clean[df_clean['孕妇代码'] == mother]
            week_mean = mother_data['孕周_数值'].mean()
            mask = (mother_data['孕周_数值']>=week_mean-0.5)&(mother_data['孕周
_数值'] <= week_mean + 0.5)
            if mask.sum() >=2:
                errors.append(mother_data[mask]['Y 染色体浓度'].std())
        return np.mean(errors) if errors else 0.005

    error_std = estimate_detection_error(df_clean)
    print(f"\n 检测误差估计结果")
    print(f"Y 染色体浓度检测误差标准差：{error_std:.4f}（{error_std * 100:.2f}%）
")

    # 模拟误差对各组风险的影响（蒙特卡洛 1000 次）
    print(f"\n 检测误差对各组风险的影响 蒙特卡洛")
    simulation_results = []
    n_sim = 1000
    for group_name, group_data in df_clean.groupby('BMI_最终分组'):
        group_best=best_time_df[best_time_df['BMI 分组'] == group_name]
        best_week=group_best['最佳时点(周)'].iloc[0]
        original_loss=group_best['风险损失'].iloc[0]
        sim_losses=[]
        sim_pass_rates=[]
        for _ in range(n_sim):
            sim_data=group_data.copy()
            sim_data['Y 染色体浓度']=sim_data['Y 染色体浓度']+np.random.normal(0,
error_std,len(sim_data))
            sim_pass= jian_ce_da_biao_lv(sim_data,best_week)
            sim_loss =risk_loss(best_week,sim_data)
            sim_pass_rates.append(sim_pass)
            sim_losses.append(sim_loss)
```

```
            mean_sim_loss=np.mean(sim_losses)
            risk_increase=(mean_sim_loss - original_loss) / original_loss * 100 if
original_loss != 0 else 0
            simulation_results.append({'BMI 分组':group_name,'原始风险
':round(original_loss, 3),
                '模拟风险均值':round(mean_sim_loss, 3),'风险增加率':round(risk_increase,
2),'模拟达标率均值': round(np.mean(sim_pass_rates), 3)})

        sim_df = pd.DataFrame(simulation_results)
        print(sim_df.to_string(index=False))



        plt.figure(figsize=(12, 6))
        plt.plot(best_time_df['BMI 分组'], best_time_df['最佳时点(周)'], 'o-',
color='#1f77b4', linewidth=2, markersize=8)

        for i, (_, row) in enumerate(best_time_df.iterrows()):
            plt.text(i,row['最佳时点(周)'] + 0.3,f'样本量:{row["样本量"]}\n 达标率:{row["
达标率"]:.0%}',ha='center', fontsize=9, va='bottom')

        plt.xlabel('BMI 分组',fontsize=12)
        plt.ylabel('最佳 NIPT 时点（周）',fontsize=12)
        plt.title('相邻互补调整后孕妇最佳 NIPT 时点分布',fontsize=14,fontweight='bold')
        plt.ylim(constraints['孕周下限'] - 0.5,15)
        plt.grid(axis='y', alpha=0.3, linestyle='--')
        plt.savefig('调整后最佳 NIPT 时点分布.png', dpi=300, bbox_inches='tight')
        plt.close()

        high_bmi_threshold = 34
        high_bmi_groups = [g for g in adjusted_bmi_labels if g.startswith('[36,')]  # 过滤
以 '[36,' 开头的组
        if len(high_bmi_groups) < 2:
            high_bmi_groups = adjusted_bmi_labels[-2:]  # 若不足 2 组，取最后 2 组

        plt.figure(figsize=(10, 6))
        for group in high_bmi_groups:
            group_data = df_clean[df_clean['BMI_最终分组'] == group]
            candidate_weeks = np.arange(12, 14.1, 0.1)
            losses = [risk_loss(w, group_data) for w in candidate_weeks]
            plt.plot(candidate_weeks, losses, linewidth=2, label=f'{group}')

        plt.axhline(y=constraints['最低达标率'], color='g', linestyle=':', alpha=0.8,label=f'最
低达标率({constraints["最低达标率"]:.0%})')
        plt.xlabel('孕周（周）', fontsize=12)
```

```
    plt.ylabel('风险损失', fontsize=12)
    plt.title('高 BMI 组 NIPT 时点-风险损失曲线（相邻互补调整后）', fontsize=14,
fontweight='bold')
    plt.legend(loc='upper right')
    plt.grid(axis='y', alpha=0.3, linestyle='--')
    plt.savefig('调整后高 BMI 时点风险曲线.png', dpi=300, bbox_inches='tight')
    plt.close()

    plt.figure(figsize=(12, 6))
    x = np.arange(len(sim_df))
    width=0.35

    bars1 =plt.bar(x-width/2,sim_df['原始风险'],width,label='原始风险
',color='#1f77b4',alpha=0.8)
    bars2= plt.bar(x+width/2,sim_df['模拟风险均值'],width,label='模拟风险（含误差）
',color='#ff7f0e',alpha=0.8)

    for bars in [bars1, bars2]:
        for bar in bars:
            height = bar.get_height()
            plt.text(bar.get_x() + bar.get_width() / 2.,height +
0.01,f'{height:.3f}',ha='center', va='bottom', fontsize=9)

    plt.xlabel('BMI 分组',fontsize=12)
    plt.ylabel('风险损失',fontsize=12)
    plt.title('检测误差对调整后分组风险损失的影响',fontsize=14,fontweight='bold')
    plt.xticks(x, sim_df['BMI 分组'])
    plt.legend()
    plt.grid(axis='y',alpha=0.3,linestyle='--')
    plt.savefig('调整后检测误差影响.png', dpi=300, bbox_inches='tight')
    plt.close()

    plt.figure(figsize=(10, 6))
    x=np.arange(len(best_time_df))
    best_weeks=best_time_df['最佳时点(周)'].values
    pass_rates=best_time_df['达标率'].values
    sample_sizes=best_time_df['样本量'].values

    bars = plt.bar(x, best_weeks,color='#1f77b4', alpha=0.8,label='最佳 NIPT 时点
（周）')

    for i, (bar, week, pr, size) in enumerate(zip(bars, best_weeks, pass_rates,
sample_sizes)):
        plt.text(bar.get_x() + bar.get_width() / 2.,bar.get_height() + 0.1,f'{week:.1f} 周
```

```
\n 达标率:{pr:.0%}\n 样本:{size} 个 ',ha='center', va='bottom', fontsize=9)

    plt.xlabel('BMI 分组',fontsize=12)
    plt.ylabel('最佳 NIPT 时点（周）',fontsize=12)
    plt.title('各组最佳 NIPT 时点分布柱状图',fontsize=14,fontweight='bold')
    plt.xticks(x, best_time_df['BMI 分组'],rotation=45)
    plt.ylim(10, 14)
    plt.grid(axis='y',alpha=0.3, linestyle='--')
    plt.legend()
    plt.tight_layout()
    plt.savefig('各组最佳 NIPT 时点柱状图.png', dpi=300, bbox_inches='tight')
    plt.close()

    plt.figure(figsize=(14,6))
    valid_single=single_group_stats[single_group_stats['样本数'] > 0]
    x = np.arange(len(valid_single))
    bmi_intervals=valid_single['BMI 区间'].values
    sample_counts=valid_single['样本数'].values

    bars=plt.bar(x, sample_counts,color='#2ca02c', alpha=0.8,label='样本数')
    for i,(bar, count) in enumerate(zip(bars, sample_counts)):
        if count>50:
            plt.text(bar.get_x() + bar.get_width() / 2.,bar.get_height() +
2,str(count),ha='center', va='bottom', fontsize=8, fontweight='bold')

    plt.xlabel('BMI 单单位区间', fontsize=12)
    plt.ylabel('样本数', fontsize=12)
    plt.title('BMI 单单位区间样本数统计图（20-21 至 50-51）', fontsize=14,
fontweight='bold')
    plt.xticks(x, bmi_intervals, rotation=45, ha='right') # ha='right' 让标签右对齐，更
易读
    plt.grid(axis='y', alpha=0.3, linestyle='--')
    plt.legend()
    plt.tight_layout()
    plt.savefig('BMI 单单位区间样本数统计图.png', dpi=300, bbox_inches='tight')
    plt.close()

    plt.figure(figsize=(8, 8))
    # 提取最终分组样本量（按分组顺序排序）
    final_group_counts = df_clean['BMI_最终分组'].value_counts().sort_index()
    group_names = final_group_counts.index.tolist()
    group_sizes = final_group_counts.values.tolist()

    group_sizes_np = np.array(group_sizes)
```

```
    group_ratios = (group_sizes_np / sum(group_sizes_np) * 100).round(1) # 计算占比（%）

    colors = ['#ff9999','#66b3ff','#99ff99', '#ffcc99', '#c2c2f0'] # 区分度高的颜色
    wedges, texts, autotexts = plt.pie(
        group_sizes_np,
        labels=group_names,colors=colors,
        autopct='%1.1f%%',startangle=90, pctdistance=0.85,labeldistance=1.1 )

    for text in texts:
        text.set_fontsize(10)
    for autotext in autotexts:
        autotext.set_fontsize(9)
        autotext.set_fontweight('bold')
        autotext.set_color('white')

    legend_labels = [f'{name}（{size} 个）' for name, size in zip(group_names, group_sizes)]
    plt.legend(wedges, legend_labels,title="BMI 分组及样本数",loc="center left",bbox_to_anchor=(1, 0, 0.5, 1))
    plt.setp(plt.gca().get_legend().get_title(), fontsize=11, fontweight='bold')
    plt.title('最终 BMI 分组样本量占比饼图',fontsize=14,fontweight='bold', pad=20)
    plt.tight_layout()
    plt.savefig('最终 BMI 分组样本量饼图.png', dpi=300,bbox_inches='tight')
    plt.close()

    plt.figure(figsize=(12, 6))
    sns.boxplot(x='BMI_最终分组', y='Y 染色体浓度', data=df_clean, palette='viridis')
    plt.title('各 BMI 分组 Y 染色体浓度箱线图', fontsize=14,fontweight='bold')
    plt.xlabel('BMI 分组',fontsize=12)
    plt.ylabel('Y 染色体浓度',fontsize=12)
    plt.xticks(rotation=45)
    plt.grid(axis='y', alpha=0.3,linestyle='--')
    plt.tight_layout()
    plt.savefig('各分组 Y 染色体浓度箱线图.png',dpi=300,bbox_inches='tight')
    plt.close()

    with pd.ExcelWriter('问题 2_相邻互补调整后最佳 NIPT 时点结果.xlsx', engine='openpyxl') as writer:
        # 新增：单单位统计和调整方案
        single_group_stats.to_excel(writer,sheet_name='BMI 单单位统计',index=False)
        adjusted_group_stats.to_excel(writer,sheet_name='调整后分组方案',index=False)
        # 原有统计结果
```

```
        final_group_stats_df.to_excel(writer,sheet_name='BMI 分组统计',index=False)
        best_time_df.to_excel(writer,sheet_name='最佳时点结果', index=False)
        sim_df.to_excel(writer,sheet_name='检测误差影响', index=False)
        # 清洗后数据（含单单位分组和最终分组）
        df_clean[['孕妇代码','BMI','BMI 单单位分组','BMI_最终分组','孕周_数值','Y 染色体浓度','聚类标签']].to_excel(
            writer, sheet_name='清洗后数据', index=False)
```

附录 4

Python 代码编写，解决问题 3

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re
import warnings
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.decomposition import PCA
from sklearn.feature_selection import RFE
from scipy.stats import t
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from typing import Tuple, List, Dict, Optional

# ----------------- 配置项 -----------------
class Config:
    KEY_COLUMNS = ['孕妇代码', '检测日期', '孕周_小数', '孕妇 BMI', '年龄', '身高', '体重','Y 染色体浓度', '重复读段的比例', '被过滤掉读段数的比例', 'GC 含量','原始读段数', '在参考基因组上比对的比例',
                   '唯一比对的读段数  ', '检测抽血次数']
    QUALIFIED_Y_THRESHOLD=0.04
    GC_NORMAL_RANGE=(0.4, 0.6)
    VIF_THRESHOLD=5  # VIF 阈值

    BMI_CLUSTER_MAX_K=6
```

```
        DEFAULT_BEST_K =3
        # 最佳时点
        MIN_GESTATIONAL_WEEK =10
        MAX_GESTATIONAL_WEEK= 25
        WEEK_STEP =0.5
        QUALIFIED_RATIO_THRESHOLD =0.9
        SUBGROUP_QUALIFIED_THRESHOLD= 0.85
        MIN_SUBGROUP_SIZE= 5

        FIG_DPI = 300
        RESULT_FILE='NIPT 问题三结果汇总.xlsx'
        CORR_HEATMAP= '相关性热力图_优化.png'
        BMI_ELBOW_PLOT= 'BMI 聚类肘部图.png'
        BMI_SCATTER_PLOT= 'BMI 与达标时间散点图.png'  # 新增图表
        BMI_BOXPLOT='BMI 分组箱线图.png'
        BEST_TIME_RISK_BAR='最佳时间与风险条形图.png'  # 新增图表
        ERROR_BOXPLOT='误差分组达标时间箱线图.png'
        ERROR_IMPACT_PLOT='误差对最佳时点的影响.png'
        VIF_PLOT='变量方差膨胀因子.png'
        PCA_VARIANCE= '主成分方差解释率.png'

    def init():

        warnings.filterwarnings('ignore')
        plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'WenQuanYi Zen
Hei', 'DejaVu Sans']
        plt.rcParams['axes.unicode_minus'] = False
        plt.rcParams['mathtext.fontset'] = 'dejavusans'

    def save(fig: plt.Figure, filename: str, title: Optional[str] = None) -> None:
        """保存图表到当前目录"""
        if title:
            fig.suptitle(title)
        plt.tight_layout()
        fig.savefig(filename, dpi=Config.FIG_DPI)
        plt.close(fig)

    def plot_vif(vif_df: pd.DataFrame, title: str = "变量方差膨胀因子(VIF)") ->
plt.Figure:
        """绘制 VIF 值柱状图"""
        vif_df = vif_df.sort_values("VIF", ascending=False)
        fig, ax = plt.subplots(figsize=(12, 7))
        ax.bar(vif_df["特征"], vif_df["VIF"], color='skyblue')
        ax.axhline(y=Config.VIF_THRESHOLD, color='r', linestyle='--', label=f'VIF 阈
```

```
值={Config.VIF_THRESHOLD}')
        ax.set_xlabel('变量')
        ax.set_ylabel('VIF 值')
        ax.set_title(title)
        ax.legend()
        ax.grid(True, alpha=0.3)
        plt.xticks(rotation=45, ha="right")
        return fig

    def plot_pca_variance(pca: PCA) -> plt.Figure:#绘制主成分方差解释率
        fig, ax = plt.subplots(figsize=(10, 6))
        explained_variance = pca.explained_variance_ratio_
        cumulative_variance = np.cumsum(explained_variance)
        components = range(1, len(explained_variance) + 1)
        ax.bar(components, explained_variance, alpha=0.7, label='单个主成分解释率')
        ax.plot(components, cumulative_variance, 'r-o', label='累积方差解释率')
        ax.axhline(y=0.9, color='g', linestyle='--', label='90%方差解释率阈值')
        ax.set_xlabel('主成分序号')
        ax.set_ylabel('方差解释率')
        ax.set_title('主成分分析(PCA)方差解释率')
        ax.set_xticks(components)
        ax.legend()
        ax.grid(True, alpha=0.3)
        return fig

    def plot_bmi_vs_time(df_pregnant: pd.DataFrame) -> plt.Figure:#绘制 BMI 与达标
时间的散点图
        fig, ax = plt.subplots(figsize=(10, 7))
        groups = df_pregnant['BMI 分组'].unique()
        colors = ['skyblue', 'lightgreen', 'lightcoral', 'orchid', 'gold', 'teal']
        for i, group in enumerate(groups):
            group_data = df_pregnant[df_pregnant['BMI 分组'] == group]
            ax.scatter(group_data['孕妇 BMI'], group_data['达标时间'],
                       label=f'BMI 分组 {group}', color=colors[i], alpha=0.6, s=30)

        z = np.polyfit(df_pregnant['孕妇 BMI'], df_pregnant['达标时间'], 1)
        p = np.poly1d(z)
        ax.plot(df_pregnant['孕妇 BMI'], p(df_pregnant['孕妇 BMI']), 'k--',
linewidth=1.5)

        ax.set_xlabel('孕妇 BMI')
        ax.set_ylabel('达标时间 (周)')
        ax.set_title('BMI 与达标时间的关系散点图')
        ax.legend()
```

```
        ax.grid(True, alpha=0.3)
        return fig

    def plot_bmi_group_boxplot(df_pregnant: pd.DataFrame) -> plt.Figure:

        fig, ax = plt.subplots(figsize=(10, 7))
        df_pregnant.boxplot(column='孕妇 BMI',by='BMI 分组',grid=False, ax=ax)
        ax.set_xlabel('BMI 分组')
        ax.set_ylabel('BMI 值')
        ax.set_title('各 BMI 分组的 BMI 分布箱线图')
        plt.suptitle('')  # 去除默认标题
        return fig
    #绘制不同 BMI 误差子组箱线图
    def plot_error_subgroup_boxplot(df_pregnant: pd.DataFrame) -> plt.Figure:

        fig, ax = plt.subplots(figsize=(12, 8))
        df_pregnant.boxplot(column='达标时间', by=['BMI 分组', '误差分组'],
grid=False, ax=ax)
        ax.set_xlabel('BMI 分组-误差分组')
        ax.set_ylabel('达标时间 (周)')
        ax.set_title('不同 BMI 分组和误差分组的达标时间分布箱线图')
        plt.suptitle('')
        plt.xticks(rotation=45,ha='right')
        return fig

    def plot_best_time_risk_bar(best_timepoints_df: pd.DataFrame) -> plt.Figure:  #绘
制最佳时间与风险条形图

        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
        ax1.bar(best_timepoints_df['BMI 分组'], best_timepoints_df['最佳时点'],
color='skyblue', zorder=3)
        ax1.set_xlabel('BMI 分组')
        ax1.set_ylabel('最佳 NIPT 时点 (周)')
        ax1.set_title('各 BMI 分组最佳 NIPT 时点')
        ax1.set_xticks(best_timepoints_df['BMI 分组'])
        for i, v in enumerate(best_timepoints_df['最佳时点']):
            ax1.text(i,v + 0.1, f'{v:.1f}', ha='center', fontsize=9)

        ax2.bar(best_timepoints_df['BMI 分组'], best_timepoints_df['平均风险'],
color='lightcoral', zorder=3)
        ax2.set_xlabel('BMI 分组')
        ax2.set_ylabel('平均风险')
        ax2.set_title('各 BMI 分组平均风险')
        ax2.set_xticks(best_timepoints_df['BMI 分组'])
```

```
        ax2.set_ylim(0, 0.5)
        for i, v in enumerate(best_timepoints_df['平均风险']):
            ax2.text(i, v + 0.02, f'{v:.2f}', ha='center', fontsize=9)

        # 添加网格线
        for ax in [ax1, ax2]:
            ax.grid(True, alpha=0.3, axis='y', zorder=0)
            ax.spines['top'].set_visible(False)
            ax.spines['right'].set_visible(False)
            ax.spines['left'].set_visible(False)

        return fig

    def yunzhou(week_str: str) -> float:
        if pd.isna(week_str):return np.nan
        match=re.match(r'(\d+)w(?:\+(\d+))?', str(week_str))
        if not match:return np.nan
        weeks, days=match.groups()
        return int(weeks)+(int(days) if days else 0)/7

    def yun_fu_zui_zao_da_biao(group: pd.DataFrame) -> Optional[float]:
        """获取每个孕妇最早达标时间"""
        qualified = group[group['Y 染色体浓度'] >=
Config.QUALIFIED_Y_THRESHOLD]
        return qualified['孕周_小数'].min() if not qualified.empty else np.nan

    def data(file_path: str) -> pd.DataFrame:
        """加载并预处理数据，返回每个孕妇的唯一记录"""
        try:
            df=pd.read_excel(file_path,sheet_name="男胎检测数据")
            print(f"原始数据总行数: {len(df)}")
        except Exception as e:
            raise ValueError(f"读取文件'{file_path}'失败: {e}")

        df_male=df[df['Y 染色体浓度'].notna()].copy()
        print(f"筛选出男胎数据条数：{len(df_male)}")

        df_male['孕周_小数']=df_male['检测孕周'].apply(yunzhou)
        df_male.dropna(subset=['孕周_小数'], inplace=True)

        df_male['检测日期']=pd.to_datetime(df_male['检测日期'],errors='coerce')
        df_male.dropna(subset=['检测日期'],inplace=True)
        df_male.sort_values(['孕妇代码', '检测日期'], inplace=True)
```

```
        qualified_time_map=df_male.groupby('孕妇代码
').apply(yun_fu_zui_zao_da_biao)
        df_male['达标时间']=df_male['孕妇代码'].map(qualified_time_map)

        df_qualified=df_male.dropna(subset=['达标时间']).copy()
        print(f"有达标时间的孕妇数据条数：{len(df_qualified)}")

        df_qualified['GC 异常'] = ((df_qualified['GC 含量'] <
Config.GC_NORMAL_RANGE[0]) |(df_qualified['GC 含量'] >
Config.GC_NORMAL_RANGE[1])).astype(int)
        df_qualified['误差评分'] = (df_qualified['重复读段的比例'] +df_qualified['被
过滤掉读段数的比例'] +(1 - df_qualified['在参考基因组上比对的比例'])
+df_qualified['GC 异常'] * 0.1)

        # 去重
        df_pregnant = df_qualified.loc[df_qualified.groupby('孕妇代码')['检测日期
'].idxmax()]
        df_pregnant = df_pregnant.drop_duplicates('孕妇代码').reset_index(drop=True)
        print(f"去重后孕妇数量：{len(df_pregnant)}")

        bmi_anomalies = df_pregnant[(df_pregnant['孕妇 BMI'] < 15) | (df_pregnant['孕
妇 BMI'] > 40)].shape[0]
        if bmi_anomalies > 0:
            print(f"检测到{bmi_anomalies}条异常 BMI 数据（<15 或>40）")

        return df_pregnant

    def analyze_influencing_factors(df_pregnant: pd.DataFrame) -> None:
        """分析影响因素，处理多重共线性并建立回归模型"""
        factors=['孕妇 BMI', '年龄', '身高', '体重', 'Y 染色体浓度', '重复读段的比例','
被过滤掉读段数的比例', 'GC 含量', '原始读段数', '在参考基因组上比对的比例','唯一
比对的读段数  ', '误差评分']
        target='达标时间'

        corr_matrix = df_pregnant[factors + [target]].corr()
        print("\n 相关性矩阵")
        print(corr_matrix[target].sort_values(ascending=False))

        # VIF 计算
        X=df_pregnant[factors].copy().dropna()
        y=df_pregnant.loc[X.index, target]

        vif_data=pd.DataFrame()
        vif_data["特征"]=X.columns
```

```
        vif_data["VIF"]=[variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]
        print("\nVIF")
        print(vif_data.sort_values("VIF", ascending=False))
        save(plot_vif(vif_data), Config.VIF_PLOT)

        high_vif_vars= vif_data[vif_data["VIF"] > Config.VIF_THRESHOLD]["特征
"].tolist()
        low_vif_vars =vif_data[vif_data["VIF"] <= Config.VIF_THRESHOLD]["特征
"].tolist()
        X_processed =X[low_vif_vars].copy()
        if high_vif_vars:
            print(f"\n检测到高共线性变量: {high_vif_vars}，进行主成分分析降维...")

            scaler = StandardScaler()
            X_high_vif_scaled = scaler.fit_transform(X[high_vif_vars])

            pca = PCA(n_components=0.9)  # 保留 90%的方差
            pca_result = pca.fit_transform(X_high_vif_scaled)

            print(f"PCA 保留了 {pca.n_components_} 个主成分，解释了
{sum(pca.explained_variance_ratio_):.2%} 的方差。")
            save(plot_pca_variance(pca), Config.PCA_VARIANCE)

            for i in range(pca.n_components_):
                X_processed[f'PCA_{i + 1}']=pca_result[:, i]

        model = LinearRegression()
        n_features=max(1, X_processed.shape[1] // 3)
        selector=RFE(model, n_features_to_select=n_features, step=1)
        selector.fit(X_processed, y)

        selected_features=X_processed.columns[selector.support_]
        print(f"\n 逐步回归选中的变量: {selected_features.tolist()}")

        # 使用 statsmodels 拟合最终模型
        X_final = sm.add_constant(X_processed[selected_features])
        final_model = sm.OLS(y, X_final).fit()

        print("\n=== 多元线性回归结果 ===")
        print(final_model.summary())

    def find_optimal_k(inertias: List[float], k_range: range) -> int:#肘部法
        if len(inertias) < 2:
```

```
            return Config.DEFAULT_BEST_K

        diffs=np.diff(inertias)
        diff_ratios = np.diff(diffs)

        optimal_idx = np.argmax(abs(diff_ratios)) + 1   # +1  补偿 diff 导致的索引偏移
        return k_range[optimal_idx] if optimal_idx < len(k_range) else
Config.DEFAULT_BEST_K

    def bmi_clustering(df_pregnant: pd.DataFrame) -> Tuple[pd.DataFrame,
pd.DataFrame]:#K-means 聚类
        X_bmi= df_pregnant[['孕妇 BMI']].values
        scaler= StandardScaler()
        X_bmi_scaled=scaler.fit_transform(X_bmi)

        inertias=[]
        k_range=range(2, Config.BMI_CLUSTER_MAX_K + 1)
        for k in k_range:
            kmeans =KMeans(n_clusters=k, random_state=42, n_init=10)
            kmeans.fit(X_bmi_scaled)
            inertias.append(kmeans.inertia_)

        best_k=find_optimal_k(inertias, k_range)
        print(f"通过肘部法则确定的最佳 BMI 聚类数量: {best_k}")

        fig,ax=plt.subplots(figsize=(8, 5))
        ax.plot(k_range, inertias, 'bo-')
        ax.axvline(x=best_k, color='r', linestyle='--', label=f'最佳 K={best_k}')
        ax.set_xlabel('聚类数量 K')
        ax.set_ylabel('簇内平方和（Inertia）')
        ax.set_title('BMI 聚类肘部法则图')
        ax.grid(True, alpha=0.3)
        ax.legend()
        save(fig,Config.BMI_ELBOW_PLOT)

        kmeans=KMeans(n_clusters=best_k, random_state=42, n_init=10)
        df_pregnant['BMI 分组']=kmeans.fit_predict(X_bmi_scaled)

        group_bmi_mean=df_pregnant.groupby('BMI 分组')['孕妇
BMI'].mean().sort_values()
        group_mapping={old: new for new, old in enumerate(group_bmi_mean.index)}
        df_pregnant['BMI 分组']=df_pregnant['BMI 分组'].map(group_mapping)

        group_stats=df_pregnant.groupby('BMI 分组').agg({
```

```
            '孕妇 BMI':['count','min','mean','max','std'],
            '达标时间':['mean','std']
        }).round(4)
        group_stats.columns=['孕妇数量', 'BMI 最小值', 'BMI 平均值', 'BMI 最大值',
'BMI 标准差','平均达标时间', '达标时间标准差']
        print("\nBMI 分组结果")
        print(group_stats)

        # BMI 与达标时间散点图
        scatter_fig = plot_bmi_vs_time(df_pregnant)
        save(scatter_fig, Config.BMI_SCATTER_PLOT)

        # BMI 分组箱线图
        boxplot_fig = plot_bmi_group_boxplot(df_pregnant)
        save(boxplot_fig, Config.BMI_BOXPLOT)

        return df_pregnant, group_stats

    def calculate_risk(gestational_week: float) -> float:
        if gestational_week <= 12: return 0.2
        if 12 < gestational_week <= 27: return 0.2 + (gestational_week - 12) * 0.02
        return 1.0

    def find_best_timepoint_for_group(group_data, threshold, min_w, max_w, step) ->
Optional[pd.DataFrame]:#找到最佳检测时点
        if len(group_data) == 0: return None
        timepoints =np.arange(min_w, max_w + step, step)
        results =[]
        for tp in timepoints:
            ratio= (group_data['达标时间'] <= tp).sum() / len(group_data)
            if ratio>=threshold:
                results.append({'时点': tp,'达标比例': ratio,'平均风险
':calculate_risk(tp)})
        return pd.DataFrame(results) if results else None

    def find_best_timepoint(df_pregnant: pd.DataFrame) -> pd.DataFrame:
        #找到最佳 NIPT 检测时点
        best_timepoints = []
        for group in sorted(df_pregnant['BMI 分组'].unique()):
            group_data=df_pregnant[df_pregnant['BMI 分组']==group]
            results_df=find_best_timepoint_for_group(group_data,
Config.QUALIFIED_RATIO_THRESHOLD,
Config.MIN_GESTATIONAL_WEEK,Config.MAX_GESTATIONAL_WEEK,
Config.WEEK_STEP)
```

```
            if results_df is not None and not results_df.empty:
                best_tp=results_df.loc[results_df['平均风险'].idxmin()]
                best_timepoints.append({'BMI 分组': group, '最佳时点': best_tp['时点'],
'达标比例': best_tp['达标比例'],'平均风险': best_tp['平均风险'], '孕妇数量':
len(group_data)})
        best_df = pd.DataFrame(best_timepoints)
        print("各 BMI 分组最佳 NIPT 时点")
        print(best_df.round(4))

        bar_fig=plot_best_time_risk_bar(best_df)
        save(bar_fig,Config.BEST_TIME_RISK_BAR)

        return best_df

    def analyze_error_impact(df_pregnant: pd.DataFrame) -> Tuple[pd.DataFrame,
pd.DataFrame]:#分析检测误差对达标时间和最佳时点的影响
        df_pregnant['误差分组'] = pd.qcut(df_pregnant['误差评分'], 3, labels=['低误差
','中误差','高误差'])
        error_stats = df_pregnant.groupby(['BMI 分组','误差分组'])['达标时间
'].agg(['count','mean','std']).round(4)
        error_stats.columns = ['孕妇数量','平均达标时间','达标时间标准差']
        print("误差分组对达标时间的影响")
        print(error_stats)

        # 误差子组箱线图
        error_boxplot_fig=plot_error_subgroup_boxplot(df_pregnant)
        save(error_boxplot_fig,Config.ERROR_BOXPLOT)

        sub_best_timepoints=[]
        for bmi_group in sorted(df_pregnant['BMI 分组'].unique()):
            for error_group in ['低误差','中误差','高误差']:
                sub_data = df_pregnant[(df_pregnant['BMI 分组'] == bmi_group) &
(df_pregnant['误差分组'] == error_group)]
                if len(sub_data)<Config.MIN_SUBGROUP_SIZE: continue

                results_df=find_best_timepoint_for_group(sub_data,Config.SUBGROU
P_QUALIFIED_THRESHOLD,Config.MIN_GESTATIONAL_WEEK,Config.MAX_G
ESTATIONAL_WEEK, Config.WEEK_STEP)

                if results_df is not None and not results_df.empty:
                    best_tp = results_df.loc[results_df['平均风险'].idxmin()]
                    sub_best_timepoints.append({'BMI 分组':bmi_group,'误差分组
':error_group,'时点':best_tp['时点'],
                        '达标比例': best_tp['达标比例'],'平均风险':best_tp['平均风险
```

```
']})

        sub_best_df = pd.DataFrame(sub_best_timepoints)
        print("\n 各 BMI-误差亚组最佳时点 ")
        print(sub_best_df.round(4))
        return error_stats, sub_best_df

    def summarize_results(bmi_group_stats: pd.DataFrame,best_timepoints_df:
pd.DataFrame,sub_best_df: pd.DataFrame,error_stats: pd.DataFrame):#汇总到 Excel

        bmi_ranges = bmi_group_stats.apply(lambda row: f"[{row['BMI 最小值
']:.1f},{row['BMI 最大值']:.1f}]", axis=1).rename('BMI 区间')
        summary_df = pd.concat([bmi_group_stats, bmi_ranges], axis=1).reset_index()
        summary_df.rename(columns={'BMI 平均值': '平均 BMI'}, inplace=True)
        final_result = pd.merge(summary_df,best_timepoints_df.drop(columns=['孕妇
数量']),on='BMI 分组')

        output_columns = ['BMI 分组','BMI 区间','平均 BMI', '孕妇数量','最佳时点','
达标比例', '平均风险']
        final_output = final_result[output_columns]

        print("\n 最终结果汇总 ")
        print(final_output.round(4))

        try:
            with pd.ExcelWriter(Config.RESULT_FILE,engine='openpyxl') as writer:
                final_output.to_excel(writer,sheet_name='BMI 分组与最佳时点
',index=False)
                sub_best_df.to_excel(writer,sheet_name='误差亚组最佳时点
',index=False)
                bmi_group_stats.to_excel(writer,sheet_name='BMI 分组统计')
                error_stats.to_excel(writer,sheet_name='误差对达标时间影响')
            print(f"结果已保存到: {Config.RESULT_FILE}")
        except Exception as e:
            print(f"保存结果失败: {str(e)}")

    # ----------------  主函数  ----------------
    def main(file_path: str = "附件.xlsx"):
        init()
        df_pregnant=data(file_path)
        analyze_influencing_factors(df_pregnant.copy())
        df_pregnant,bmi_group_stats=bmi_clustering(df_pregnant.copy())
        best_timepoints_df=find_best_timepoint(df_pregnant.copy())
        error_stats,sub_best_df=analyze_error_impact(df_pregnant.copy())
```

```
        summarize_results(bmi_group_stats, best_timepoints_df, sub_best_df,
error_stats)

    if __name__ == "__main__":
        main(file_path="附件.xlsx")
```

附录 5

Python 代码编写，解决问题 4

```
    import pandas as pd
    import numpy as np
    import matplotlib.pyplot as plt
    import seaborn as sns
    import warnings
    from sklearn.metrics import
(precision_recall_curve,average_precision_score,precision_score,recall_score,
                f1_score, roc_auc_score,confusion_matrix,
accuracy_score,classification_report,roc_curve)
    from sklearn.preprocessing import StandardScaler, OneHotEncoder
    from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
    from sklearn.model_selection import GroupKFold, GridSearchCV, train_test_split
    from sklearn.pipeline import Pipeline
    from scipy.stats import norm
    from typing import Union, List, Dict, Tuple
    import xgboost as xgb

    plt.switch_backend('Agg')
    warnings.filterwarnings('ignore')
    plt.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'WenQuanYi Zen Hei']
    plt.rcParams['axes.unicode_minus'] = False
    plt.rcParams['mathtext.fontset'] = 'dejavusans'

    def yunzhou(week_str:Union[str,float,int]) -> float:
        if pd.isna(week_str):return np.nan
        week_str=str(week_str).strip()
        if 'w' in week_str:
            try:
                w_part, rest = week_str.split('w', 1)
                weeks = int(w_part)
                if '+' in rest:
                    d_part = rest.split('+')[1].strip()
                    weeks += int(d_part) / 7
                return round(weeks, 3)
            except (ValueError, IndexError):
```

```
                warnings.warn(f"无法解析孕周格式：{week_str}，返回 NaN")
                return np.nan
        try:
            return round(float(week_str), 3)
        except ValueError:
            warnings.warn(f"无法解析孕周格式：{week_str}，返回 NaN")
            return np.nan

    def huaiyuncishu(count:Union[str, int]) -> int:
        if pd.isna(count):return np.nan
        count_str=str(count).strip()
        return 3 if count_str == '≥3' else int(count_str)

    def p_ivf(ivf_str: str) -> int:
        if pd.isna(ivf_str):return np.nan
        ivf_str=str(ivf_str).strip()
        return 1 if any(keyword in ivf_str for keyword in ['IVF','试管婴儿','IUI']) else 0

    def process_target(abnormal_str: str) -> int:
        if pd.isna(abnormal_str) or str(abnormal_str).strip() == '':return 0
        abnormal_types=str(abnormal_str).strip().split('T')
        for typ in abnormal_types:
            if typ in ['13','18','21']:return 1
        return 0

    def create_features(df: pd.DataFrame) -> pd.DataFrame:
        """创建测序质量派生特征：有效读段、Z 值可靠性、GC 偏离度等"""
        df = df.copy()
        df['有效读段比例']=1-df['被过滤掉读段数的比例']
        df['有效读段数_估计']=df['原始读段数']*df['有效读段比例']
        df['原始读段数_对数']=np.log1p(df['原始读段数'])
        df['唯一比对的读段数_比例']=df['唯一比对的读段数']/df['原始读段数']
        df['唯一比对_有效比例']= df['唯一比对的读段数']/df['有效读段数_估计']

        for chrom in ['13','18','21','X']:
            df[f'{chrom}号 Z 值_绝对值']=np.abs(df[f'{chrom}号染色体的 Z 值'])
        for chrom in ['13','18','21']:
            df[f'{chrom}号 Z 值_可靠度']= df[f'{chrom}号 Z 值_绝对值'] *df['有效读
段比例']

        normal_gc_range=(0.4,0.6)
        df['全局 GC_偏离度']=np.abs(df['GC 含量']-np.mean(normal_gc_range))
        df['全局 GC_异常标志']=((df['GC 含量']<normal_gc_range[0])|(df['GC 含量
'] >normal_gc_range[1])).astype(int)
```

```
        for chrom in ['13','18','21']:
            df[f'{chrom}号 GC_偏离度'] =np.abs(df[f'{chrom}号染色体的 GC 含量']-
np.mean(normal_gc_range))

        df['X 染色体浓度_标准化']= (df['X 染色体浓度']-df['X 染色体浓度
'].mean())/df['X 染色体浓度'].std()
        df['XZ_与_GC 交互']= df['X 号 Z 值_绝对值']*df['全局 GC_偏离度']
        return df

    def encode_features(df: pd.DataFrame) -> pd.DataFrame:
        df=df.copy()
        df['怀孕次数_分组']=pd.cut(df['怀孕次数_数值'],bins=[0,1,2,np.inf],labels=['1
次', '2 次', '≥3 次'], right=False)
        ohe=OneHotEncoder(sparse_output=False, drop='first',dtype=int)
        ohe_features=ohe.fit_transform(df[['怀孕次数_分组']])
        ohe_df =pd.DataFrame(ohe_features,columns=[f'怀孕次数_{label}' for label in
ohe.categories_[0][1:]],index=df.index)
        df=pd.concat([df, ohe_df], axis=1)
        return df

    def data(file_path: str, sheet_name: str = '女胎检测数据') -> pd.DataFrame:
        """完整预处理流程：数据加载→格式转换→特征工程→缺失值填充"""
        core_cols =['序号', '孕妇代码', '年龄', '身高', '体重', '孕妇 BMI', '检测孕周',
            '怀孕次数', '生产次数', 'IVF 妊娠', '原始读段数', '在参考基因组上比对的
比例','重复读段的比例', '唯一比对的读段数', 'GC 含量', '被过滤掉读段数的比例',
            '13 号染色体的 Z 值', '18 号染色体的 Z 值', '21 号染色体的 Z 值', 'X 号染
色体的 Z 值', 'X 染色体浓度','13 号染色体的 GC 含量', '18 号染色体的 GC 含量', '21
号染色体的 GC 含量', '染色体的非整倍体']
        df = pd.read_excel(file_path, sheet_name=sheet_name, usecols=core_cols,
engine='openpyxl')
        print(f"原始数据：{df.shape} 行×列，孕妇数量：{df['孕妇代码'].nunique()}")
        df['检测孕周_数值']=df['检测孕周'].apply(yunzhou)
        df['怀孕次数_数值']=df['怀孕次数'].apply(huaiyuncishu)
        df['生产次数_数值']= df['生产次数'].apply(lambda x: int(x) if pd.notna(x) else
np.nan)
        df['IVF_编码'] =df['IVF 妊娠'].apply(p_ivf)
        df['目标变量_异常']= df['染色体的非整倍体'].apply(process_target)
        df = create_features(df)
        df = encode_features(df)

        numeric_cols=[col for col in df.columns if df[col].dtype in [np.float64, np.int64]
and col not in ['目标变量_异常']]
        for col in numeric_cols:
            if df[col].isnull().sum() > 0:
```

```
                df[col] = df.groupby('孕妇代码')[col].transform(lambda x:
x.fillna(x.mean()))
                df[col].fillna(df[col].median(), inplace=True)
        print(f"\n 预处理后目标变量分布：正常={df['目标变量_异常
'].value_counts()[0]}，异常={df['目标变量_异常'].value_counts()[1]}")
        return df

    def fenzu(df: pd.DataFrame, test_size: float = 0.3, random_state: int = 42) -> Tuple[
        pd.DataFrame,pd.DataFrame]:   #按孕妇分组划分训练/测试集，确保同孕妇
样本在同一集合
        unique_patients=df['孕妇代码'].unique()
        train_patients,test_patients=train_test_split(unique_patients, test_size=test_size,
random_state=random_state,
            stratify=df.groupby('孕妇代码')['目标变量_异常'].max())
        df_train=df[df['孕妇代码'].isin(train_patients)].copy()
        df_test=df[df['孕妇代码'].isin(test_patients)].copy()
        print(f"训练集：{df_train.shape[0]}样本，{df_train['孕妇代码'].nunique()}孕
妇，异常比例{df_train['目标变量_异常'].mean():.1%}")
        print(f"测试集：{df_test.shape[0]}样本，{df_test['孕妇代码'].nunique()}孕妇，
异常比例{df_test['目标变量_异常'].mean():.1%}")
        return df_train, df_test

    def calculate_thresholds(y_true: np.ndarray, y_proba: np.ndarray) -> Tuple[Dict,
pd.DataFrame]:
        """双阈值优化：高召回率阈值（≥95%）和 F2 最优阈值"""
        precision,recall,thresholds=precision_recall_curve(y_true, y_proba)
        precision,recall=precision[:-1], recall[:-1]
        beta=2
        f2_scores=(1+beta**2)*(precision*recall)/(beta**2*precision+recall+1e-7)
        high_recall_indices=np.where(recall>=0.95)[0]
        high_recall_threshold=thresholds[np.argmax(precision[high_recall_indices])] if
high_recall_indices.size > 0 else \
        thresholds[np.argmax(f2_scores)]
        f2_opt_threshold = thresholds[np.argmax(f2_scores)]

        return {'高召回率阈值':high_recall_threshold,'F2 最优阈值
':f2_opt_threshold},pd.DataFrame({'阈值': thresholds, '精确率': precision, '召回率':
recall, 'F2 分数': f2_scores}).round(4)

    def train_model(df_train: pd.DataFrame, model_name: str) -> Tuple[Pipeline,
List[str], StandardScaler]:
        feature_cols = ['年龄','身高','体重','孕妇 BMI','检测孕周_数值','生产次数_数
值','IVF_编码','原始读段数_对数', '在参考基因组上比对的比例', '重复读段的比例', '
唯一比对的读段数_比例',
```

```
            '有效读段比例', '全局 GC_偏离度', '全局 GC_异常标志','13 号 Z 值_可靠度', '18 号 Z 值_可靠度', '21 号 Z 值_可靠度', 'X 号 Z 值_绝对值',
            '13 号 GC_偏离度', '18 号 GC_偏离度','21 号 GC_偏离度','X 染色体浓度_标准化', 'XZ_与_GC 交互','怀孕次数_2 次', '怀孕次数_≥3 次']
        feature_cols=[col for col in feature_cols if col in df_train.columns]
        X=df_train[feature_cols]
        y=df_train['目标变量_异常']
        groups=df_train['孕妇代码']
        print(f"\n 开始训练 {model_name} 模型：使用{len(feature_cols)}个特征，样本数{X.shape[0]}")
        scaler = StandardScaler()
        #pipeline = None
        #param_grid = None

        if model_name =='RandomForest':
            model= RandomForestClassifier(class_weight='balanced', random_state=42, n_jobs=-1)
            param_grid= {'model__n_estimators': [100, 200],'model__max_depth': [5, 8, 12],'model__min_samples_leaf': [1, 2, 4]}
            pipeline =Pipeline([('scaler', scaler), ('model', model)])
        elif model_name=='XGBoost':
            model=xgb.XGBClassifier(random_state=42, eval_metric='logloss', n_jobs=-1)
            param_grid={'model__n_estimators': [100, 200],'model__max_depth': [3, 5, 7],'model__learning_rate': [0.05, 0.1]}
            pipeline=Pipeline([('scaler', scaler), ('model', model)])

        elif model_name=='GradientBoosting':
            model=GradientBoostingClassifier(random_state=42,min_samples_leaf=5,subsample=0.8,validation_fraction=0.1,n_iter_no_change=5)
            param_grid= {'model__n_estimators': [100, 200],'model__max_depth': [3, 5, 7],'model__learning_rate': [0.05, 0.1, 0.2]}
            pipeline =Pipeline([('scaler', scaler), ('model', model)])
        else:
            raise ValueError("支持的模型：'RandomForest', 'XGBoost', 'GradientBoosting'")
        if pipeline is None or param_grid is None:
            raise ValueError(f"模型 {model_name} 初始化失败")
        group_cv = GroupKFold(n_splits=5)
        grid_search = GridSearchCV(
            estimator=pipeline,param_grid=param_grid,cv=group_cv.split(X, y, groups),scoring='roc_auc',n_jobs=-1,verbose=1)
        grid_search.fit(X, y)
        best_pipeline = grid_search.best_estimator_
```

```
        print(f"\n{model_name} 最优参数：{grid_search.best_params_}")
        print(f"{model_name} 交叉验证平均召回率：{grid_search.best_score_:.3f}")

        if model_name in ['RandomForest', 'XGBoost', 'GradientBoosting']:
            final_model = best_pipeline.named_steps['model']
            if hasattr(final_model, 'feature_importances_'):
                feature_importance_df = pd.DataFrame({'特征名称': feature_cols,'重要
性': final_model.feature_importances_}).sort_values('重要性',
ascending=False).round(4)
                print(f"\n{model_name} - Top10 关键特征：")
                print(feature_importance_df.head(10))
                plt.figure(figsize=(10, 6))
                sns.barplot(x='重要性',y='特征名称
',data=feature_importance_df.head(10), palette='viridis')
                plt.title(f'{model_name}-Top10 特征重要性', fontsize=12)
                plt.savefig(f'{model_name}_特征重要性.png', dpi=300)
                plt.close()
        return best_pipeline, feature_cols, scaler

    def traditional_z_rule(df: pd.DataFrame) -> np.ndarray:
        z_cols = ['13 号染色体的 Z 值', '18 号染色体的 Z 值', '21 号染色体的 Z 值']
        return ((np.abs(df[z_cols])>3).any(axis=1)).astype(int)

    def evaluate_model(y_true: np.ndarray, y_pred: np.ndarray, y_proba: np.ndarray,
model_name: str) -> Dict:
        beta=2
        precision=precision_score(y_true, y_pred, zero_division=0)
        recall=recall_score(y_true, y_pred, zero_division=0)
        f2 = (1+beta**2)*(precision * recall) / (beta ** 2 * precision + recall + 1e-7)
        return {
            '模型名称': model_name,
            '准确率': round(accuracy_score(y_true, y_pred), 3),
            '精确率': round(precision,3),
            '召回率': round(recall,3),
            'F1 分数': round(f1_score(y_true, y_pred, zero_division=0), 3),
            'F2 分数': round(f2, 3),
            'ROC-AUC': round(roc_auc_score(y_true, y_proba), 3) if y_proba is not
None else np.nan,
            'PR-AUC': round(average_precision_score(y_true, y_proba), 3) if y_proba is
not None else np.nan}

    def compare_models(df_test: pd.DataFrame, models: Dict[str, Pipeline],
feature_cols: List[str]) -> pd.DataFrame:
        """对比多个模型与传统 Z 值规则，保存 P-R 曲线和混淆矩阵"""
```

```
        X_test = df_test[feature_cols]
        y_test = df_test['目标变量_异常']
        all_metrics = []
        plot_data = {}
        colors = {'RandomForest': '#2E86AB', 'XGBoost': '#A23B72',
'GradientBoosting': '#F18F01', 'Z 值规则': '#3E8914'}

        for name, pipeline in models.items():
            y_proba = pipeline.predict_proba(X_test)[:, 1]
            thresholds, _ = calculate_thresholds(y_test, y_proba)
            y_pred_f2 = (y_proba >= thresholds['F2 最优阈值']).astype(int)
            metrics = evaluate_model(y_test, y_pred_f2, y_proba, f'{name} (F2 阈值
={thresholds["F2 最优阈值"]:.3f})')
            all_metrics.append(metrics)

            metrics_clean_name = evaluate_model(y_test, y_pred_f2, y_proba, name)
            plot_data[name] = {'proba': y_proba, 'metrics': metrics_clean_name}

        y_pred_z = traditional_z_rule(df_test)
        metrics_z = evaluate_model(y_test, y_pred_z, y_pred_z, '传统 Z 值规则
(|Z|>3)')
        all_metrics.append(metrics_z)

        metrics_z_clean = evaluate_model(y_test, y_pred_z, y_pred_z, 'Z 值规则')
        plot_data['Z 值规则'] = {'proba': y_pred_z, 'metrics': metrics_z_clean}

        compare_df = pd.DataFrame(all_metrics).sort_values('F2 分数
',ascending=False)
        print("\n 模型性能对比")
        print(compare_df.to_string(index=False))

        metrics_for_plot = pd.DataFrame([data['metrics'] for data in plot_data.values()])
        metrics_melted = metrics_for_plot.melt(id_vars='模型名称',value_vars=['准确
率','精确率','召回率','F2 分数'],var_name='指标', value_name='分数')

        plt.figure(figsize=(12, 7))
        ax = sns.barplot(data=metrics_melted, x='模型名称', y='分数', hue='指标',
palette='muted')
        plt.title('各模型核心性能指标对比', fontsize=16)
        plt.ylabel('指标得分', fontsize=12)
        plt.xlabel('模型', fontsize=12)
        plt.xticks(rotation=10, ha='right')
        plt.ylim(0, 1.05)
        plt.legend(title='性能指标', fontsize=10)
```

```
        for p in ax.patches:
            ax.annotate(f'{p.get_height():.2f}',(p.get_x() + p.get_width() / 2.,
p.get_height()),ha='center', va='center',fontsize=9, color='black',
                        xytext=(0, 5),textcoords='offset points')
        plt.tight_layout()
        plt.savefig('模型性能对比柱状图.png', dpi=300)
        plt.close()

        # P-R 曲线和 ROC 曲线合并绘制
        fig,(ax1, ax2)=plt.subplots(1,2,figsize=(16, 7))
        for name, data in plot_data.items():
            precision, recall, _ = precision_recall_curve(y_test, data['proba'])
            ax1.plot(recall, precision, lw=2, label=f'{name}
(PR-AUC={data["metrics"]["PR-AUC"]:.3f})', color=colors[name],
                     linestyle='--' if name == 'Z 值规则'else '-')
            fpr, tpr, _ = roc_curve(y_test, data['proba'])
            ax2.plot(fpr, tpr, lw=2, label=f'{name}
(ROC-AUC={data["metrics"]["ROC-AUC"]:.3f})', color=colors[name],
                     linestyle='--' if name == 'Z 值规则' else '-')
        ax1.set_title('P-R 曲线对比', fontsize=14)
        ax1.set_xlabel('召回率 (Recall) - 减少漏诊')
        ax1.set_ylabel('精确率 (Precision) - 减少误诊')
        ax1.grid(alpha=0.3)
        ax1.legend()
        ax2.plot([0, 1], [0, 1], 'k--', lw=2, label='随机猜测')
        ax2.set_title('ROC 曲线对比', fontsize=14)
        ax2.set_xlabel('假阳性率 (FPR)')
        ax2.set_ylabel('真阳性率 (TPR)')
        ax2.grid(alpha=0.3)
        ax2.legend()

        plt.tight_layout()
        plt.savefig('PR_and_ROC_curves.png', dpi=300)
        plt.close()

        best_model_name = compare_df.iloc[0]['模型名称'].split(' ')[0]
        if best_model_name in models:
            best_pipeline = models[best_model_name]
            y_proba_best = best_pipeline.predict_proba(X_test)[:, 1]
            best_thresholds, _ = calculate_thresholds(y_test, y_proba_best)
            y_pred_best = (y_proba_best >= best_thresholds['F2 最优阈值']).astype(int)

            cm = confusion_matrix(y_test, y_pred_best)
            plt.figure(figsize=(6, 4))
```

```
            sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['正常', '异常'], yticklabels=['正常', '异常'])
            plt.title(f'最优模型({best_model_name})混淆矩阵', fontsize=12)
            plt.xlabel('预测标签', fontsize=11)
            plt.ylabel('实际标签', fontsize=11)
            plt.tight_layout()
            plt.savefig(f'最优模型_{best_model_name}_混淆矩阵.png', dpi=300)
            plt.close()
        return compare_df

    def predict_single_sample(input_df: pd.DataFrame, best_pipeline: Pipeline, feature_cols: List[str],
                              thresholds: Dict) -> Dict:
        ohe = OneHotEncoder(sparse_output=False, drop='first', dtype=int)
        if '怀孕次数_分组' not in input_df.columns:
            input_df['怀孕次数_分组']=pd.cut(input_df['怀孕次数_数值'], bins=[0, 1, 2, np.inf],labels=['1 次','2 次','≥3 次'], right=False)
            ohe_features=ohe.fit_transform(input_df[['怀孕次数_分组']])
            ohe_df=pd.DataFrame(ohe_features,columns=[f'怀孕次数_{label}' for label in ['2 次', '≥3 次']],index=input_df.index)
            input_df=pd.concat([input_df, ohe_df],axis=1)

        X_input=input_df.reindex(columns=feature_cols)
        y_proba=best_pipeline.predict_proba(X_input)[0,1]
        risk_level,suggestion=get_risk_strategy(y_proba, thresholds['高召回率阈值'])

        return {'样本信息':{'孕妇代码':str(input_df['孕妇代码'].iloc[0])},
            '判定结果':{'异常概率':round(float(y_proba),3),'风险等级': risk_level,
            '判定(F2 最优)':'异常' if y_proba >= thresholds['F2 最优阈值'] else '正常'},'临床建议': suggestion}

    def get_risk_strategy(prob: float,threshold_high:float,threshold_low:float=0.3) -> Tuple[str, str]:
        if prob>=threshold_high:
            return '高风险','建议立即安排羊水穿刺等确诊检测。'
        elif threshold_low <= prob < threshold_high:
            return '中风险','建议 1-2 周后复查 NIPT，并密切监测。'
        else:
            return '低风险', '按常规产检流程即可。'

    def main(file_path: str) -> None:
        df=data(file_path)
        df_train, df_test =fenzu(df)
        models_to_train = ['RandomForest','XGBoost','GradientBoosting']
```

```
        trained_models = {}
        feature_cols = []
        for model_name in models_to_train:
            pipeline, f_cols, _ = train_model(df_train, model_name)
            trained_models[model_name] = pipeline
            if not feature_cols:
                feature_cols = f_cols
        compare_df=compare_models(df_test, trained_models, feature_cols)

        best_model_row=compare_df.iloc[0]
        best_model_name=best_model_row['模型名称'].split(' ')[0]
        best_pipeline=trained_models[best_model_name]
        print(f"最优模型为:{best_model_name} (测试集F2分数:{best_model_row['F2
分数']:.3f})")

        y_proba_best = best_pipeline.predict_proba(df_test[feature_cols])[:,1]
        best_thresholds, _ =calculate_thresholds(df_test['目标变量_异常'],
y_proba_best)
        sample_data=df_test.iloc[[0]].copy()
        sample_result=predict_single_sample(sample_data, best_pipeline, feature_cols,
best_thresholds)
        for key, val in sample_result.items():
            print(f"{key}： {val}")




    if __name__=="__main__":
        main(file_path="附件.xlsx")
```